

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: SOFTWARE ENGINEERING

Subject code: CS T55

UNIT IV

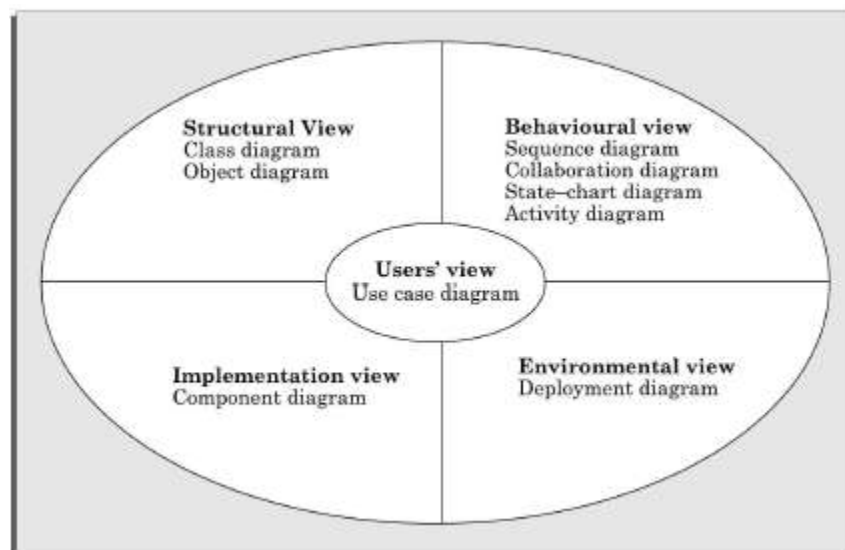
Object Modelling and Object Oriented Software development: Overview of OO concepts – UML – Use case model – Class diagrams – Interaction diagrams – Activity diagrams – state chart diagrams - Patterns – Types – Object Oriented Analysis and Design methodology – Interaction Modelling – OOD Goodness criteria.

2 Marks

1. Why do we need UML Diagram?

UML used to construct nine different types of diagrams to capture five different view of a system such as User's view, Structural view, Behavioral view, Implementation view and Environmental view.

2. Draw different types of diagrams and views supported in UML.



3. Why construct several models from different perspectives?

If a single model is made to capture all the required perspectives, then it would be as complex as the original problem, and would be of very little use.

4. What is the purpose of User's view?

The user's view captures the view of the system in terms of the functionalities offered by the system to its users.

5. Define Structural view.

The structural view defines the kinds of objects (classes) important to the understanding of the

working of a system and to its implementation. It also captures the relationships among the classes (objects).

The structural model is also called the static model, since the structure of a system does not change with time

6. Why do we need to call structural model is static?

The structure of a system does not change with time

7. Define Behavioral view.

The behavioral view captures how objects interact with each other to realize the system behavior. The system behavior captures the time-dependent (dynamic) behavior of the system.

8. What is the use of Implementation view?

This view captures the important components of the system and their dependencies.

9. Describe Environmental view?

This view models how the different components are implemented on different pieces of hardware.

10. Define Use Case Model.

The use case model for any system consists of a set of “use cases”.

Intuitively, use cases represent the different ways in which a system can be used by the users. It is an important analysis and design artifact. In contrast to all other types of UML diagrams, the user case model represents a functional or process model of a system.

11. Describe instances(scenarios)of the use case

A use case can be viewed as a set of related scenarios tied together by a common goal. The main line sequence and each of the variations are called scenarios or instances of the use case. Each scenario is a single path of user events and system activity.

12. Why do we need stereotype?

The creators of UML introduced the stereotype construct which when used to annotate a basic symbol can slightly change the meaning of a basic symbol, thereby doing away with the need to have several symbols with meanings that differ slightly from each other.

13. How to identify the Use Cases of a System?

Identification of the use cases is actor-based, in which brainstorming and reviewing the SRS document performed.

14. Give difference between Essential use cases and Real Use Case.

Essential use cases are created during early requirement elicitation. It is early problem analysis artifacts.

Real use cases describe the functionality of the system in terms of its actual current design committed to specific input/output technologies. It is a design artifact.

15. When will require factoring of use cases?

First, complex use cases need to be factored into simpler use cases. This would not only make the behavior associated with the use case much more comprehensible, but also make the corresponding interaction diagrams more tractable. Without decomposition, the interaction diagrams for complex use cases may become too large to be accommodated on a single sized (A4) paper.

Secondly, use cases need to be factored whenever there is common behavior across different use cases. Factoring would make it possible to define such behavior only once and reuse it whenever required.

16. Describe Class diagrams

A class diagram describes the static structure of a system.

Shows how a system is structured rather than how it behaves.

17. How to represent Classes?

The classes represent entities with common features that is attributes and operations. It is represented as solid outline rectangles with compartments.

18. How to represent attributes?

An attributed is named property of a class.

It represents the kind of data that an object might contain. Attributes are listed with their names, and may optionally contain specification of their type, an initial value, and constraints. Attribute names may be followed by square brackets containing a multiplicity expression.

Eg. sensorStatus[10].

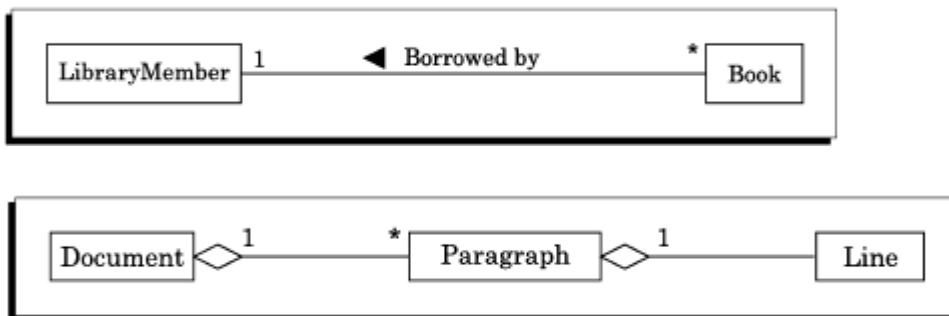
19. How to write operation?

- The operation names are left justified, in plain type, and always begin with a lower case letter. Eg. issueBook(in bookName):Boolean.
- Abstract operations are written in italics.
- The parameters of a function may have a kind specified.
- The kind may be “in” indicates the parameter is passed into the operation and “out” indicates the parameter is only returned from the operation or “inout” indicating that the parameter is used for passing data into the operation and getting result from the operation. The default is “in”

20. When will you distinguish method and operation terms?

The terms operation and method are distinguishable only when there is polymorphism. An operation is something that is supported by a class and invoked by objects of other classes. There can be multiple methods implementing the same operation.

21. Draw diagram for association and aggregation concepts.

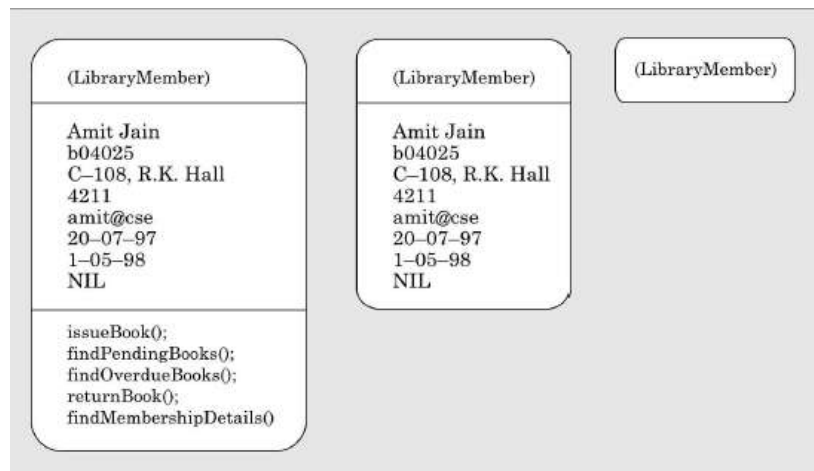


22. What is the use of constraints?

To describe the permissible set of values of an attribute and specify the pre- and post-conditions for operation.

23. Draw Object Diagram(Instance diagram) for library automation system.

It shows the snapshot of the objects in a system at a point in time.



24. Define Interaction Diagram.

A model, which is used to describe how group of objects interact among themselves through message passing to realize some behavior.

25. Give types of Interaction diagram

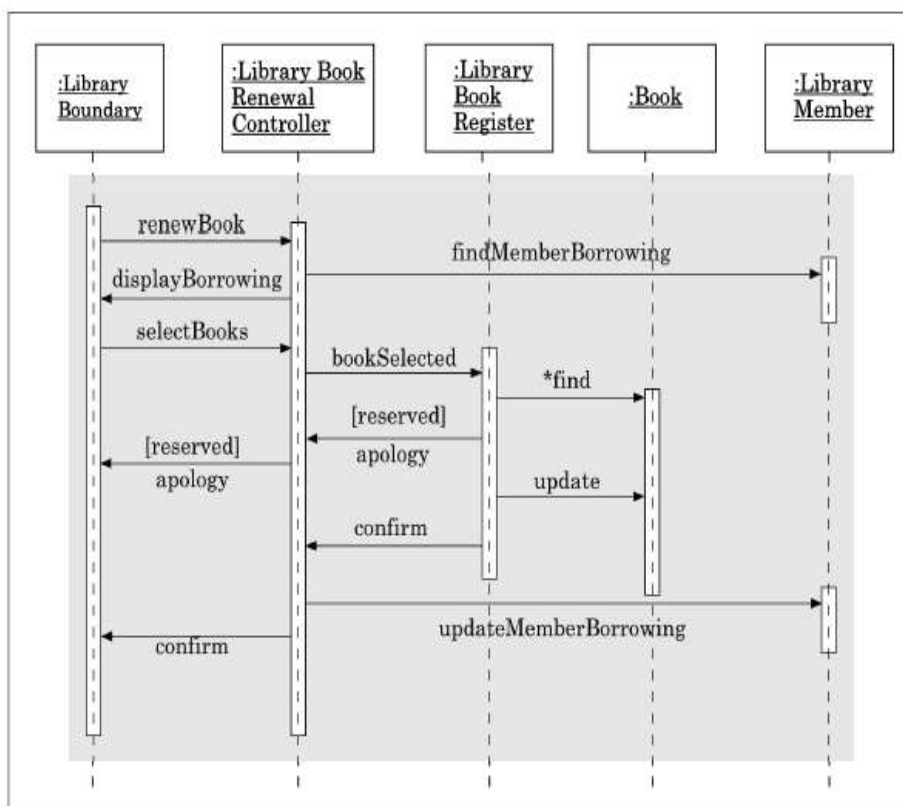
Sequence diagram and Collaboration diagram

26. What is the difference between Sequence diagram and Collaboration diagram

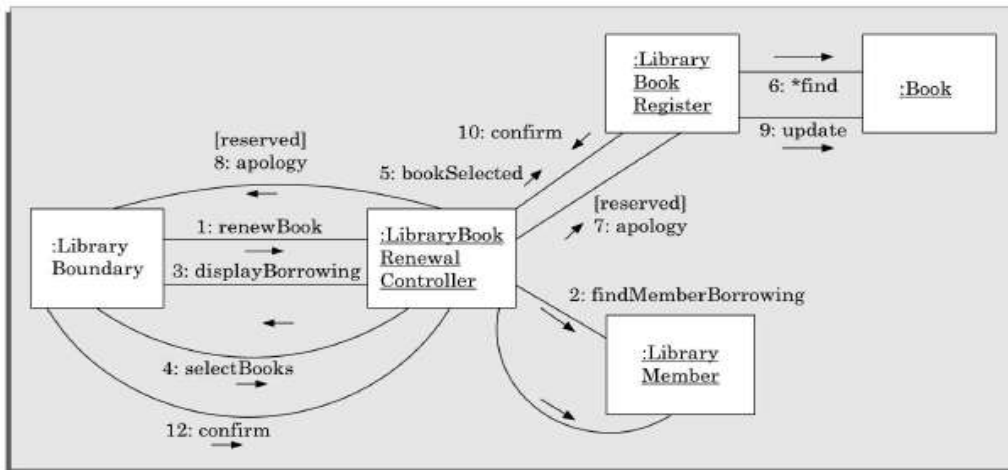
A collaboration diagram shows both structural and behavioral aspects explicitly.

A Sequence diagram shows only the behavioral aspects.

27. Draw Sequence diagram for the renew book use case



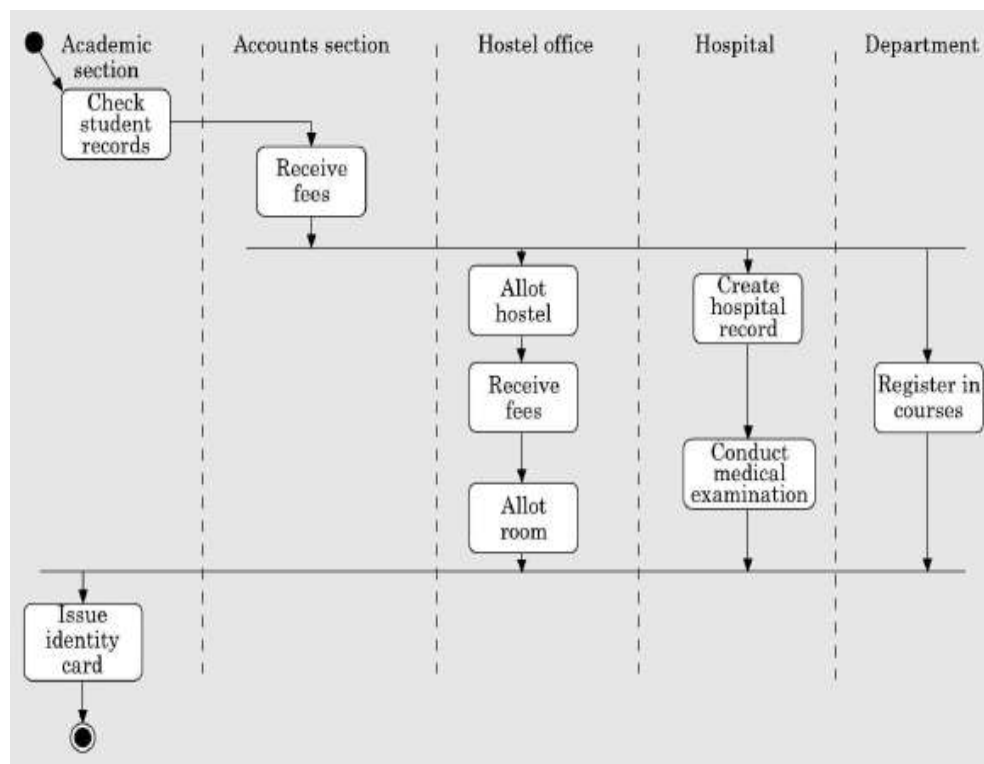
28. Draw Collaboration diagram for the renew book use case



29. What is the use of Activity diagrams?

A modeling element, which is used to focus various activities or chunks of processing and their sequence of activation. An activity is a state with an internal action and one or more outgoing transitions which automatically follow the termination of the internal activity.

30. Draw Activity diagram for student admission procedure at SMVEC.



31. Define State Chart Diagram

A modeling element, which is used to model how the state of an object changes in its life time.

32. Give basic elements of State Chart Diagram

- **Initial state:** This is represented as a filled circle
- **Final state:** This is represented by a filled circle inside a larger circle
- **State.** These are represented by rectangles with rounded corners.
- **Transition.** A transition is shown as an arrow between two states

33. Define Object-Oriented Analysis(OOA).

The term Object-Oriented Analysis refers to developing an initial model of a software product from an analysis of its requirements specification

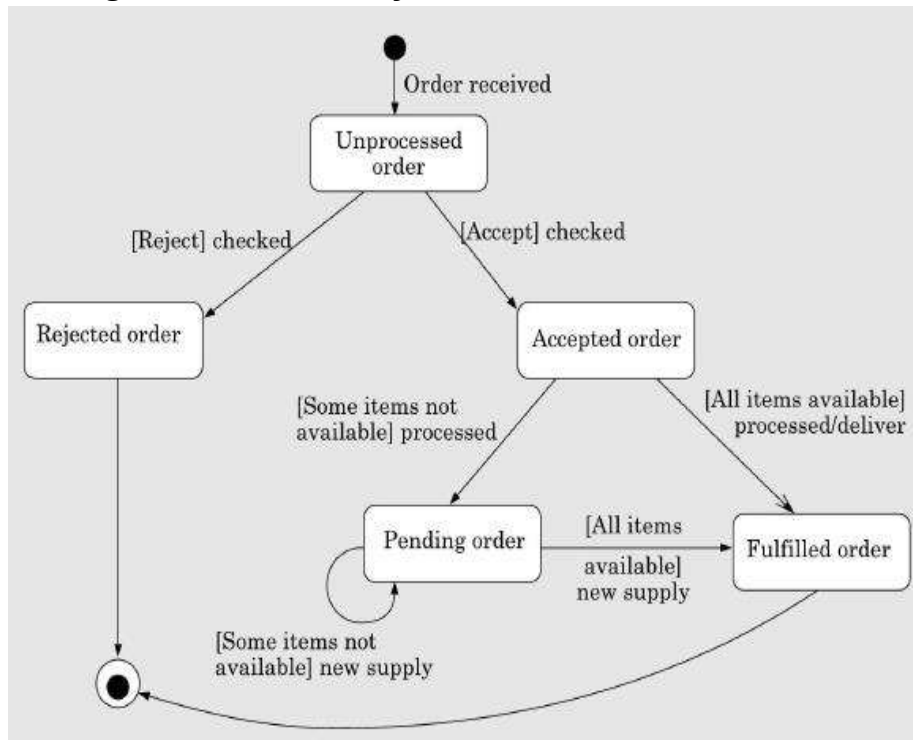
34. Define Pattern.

Design pattern are commonly accepted solutions to some problems that recur during designing different applications.

35. Give advantage of using pattern?

- The basic idea behind patterns is that if you can master a few important patterns, you can easily spot them in application development problems and effortlessly use the pattern solutions.
- Patterns can be viewed as helping designers to make certain important design decisions. At a basic level, patterns can also be viewed as well-documented building blocks for software design.

36. Draw State Chart Diagram for an order object



37. What are the parts in pattern documentation?

- The problem,
- The context in which the problem occurs,
- The solution, and
- The context within which the solution would work and would not work.

38. List out types of patterns?

- Architectural patterns
- Design patterns
- Idioms

39. Describe Idiom

An idiom describes how to implement a solution to a particular problem using the features of a given programming language.

40. What is Design patterns: A scheme for structuring the classes in a design solution and defines the interactions required among those classes.

41. What is Architectural pattern?

A very high-level design, which is used to identify and provide solutions to very large problems. It concern the overall structure of software systems

42. Give comparison of different types of patterns

The main differences are based on levels of abstraction and details they deal with.

- Architectural patterns – high level strategies that concern the overall solution to large-scale problems
- Design patterns are solutions for specific parts of medium-scale problems and recommend certain structures and behavior of the participating entities
- Idioms are paradigm-specific and language-specific programming solutions that recommend using appropriate code segments for solving low-level programming problems.

43. What is difference between pattern and algorithm

In contrast to algorithms, patterns are more concerned with aspects such as maintainability and ease of development rather than space and time efficiency.

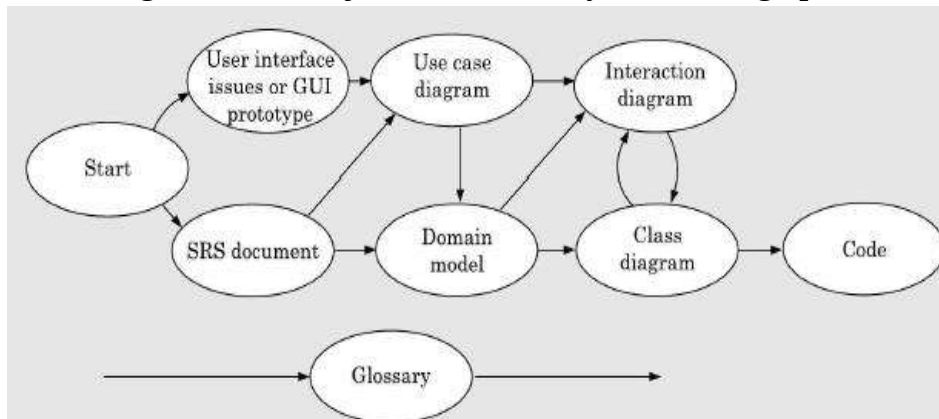
44. Give characteristics of the unified process

- Use case-driven
- Iterative

45. List out four phases of the unified process

- Inception
- Elaboration
- Construction
- Transition

46. Draw Diagram for an object-oriented analysis and design process.



47. How to develop the use case model for a given problem?

- The use case can easily be identified from the SRS document
- High level functional requirements correspond to the use cases
- An overriding principle while identifying and packaging use cases is that there should be a strong correlation between the GUI prototype, the contents of the user's manual and the use case model of the system.
- For example, the File menu may contain functions such as :load file, store file, save as, print., Each of the menu options in the top-level menu of the GUI would usually correspond to a package in the use case diagram.

48. List out common mistakes committed in use case model development

- Clutter
- Too detailed

- Omitting text description
- Overlooking some alternate scenarios

49. Define Domain(Conceptual)Modelling.

A conceptual model depicts the concepts(objects) that are easily identifiable in the problem description.

50. What are the objects identified during domain analysis?

- It contains three types of objects
 - Boundary objects(Interface objects)
 - Controller(conceptual) objects
 - Entity objects

51. Describe Boundary objects(Interface objects)

- a. objects handle the user interface
Eg **issue book** user interface
- b. The boundary objects are those with which the actors interact
- c. The boundary objects include screens, menus, forms, dialogs,etc
- d. The boundary objects are mainly responsible for evaluating user interactions through suitable graphical user interfaces(GUIs)
- e. Responsible to read inputs form the user, validate the inputs, format the outputs, and display the results.

52. Write about Controller (Conceptual) Objects?

- a. objects are entirely conceptual
Eg.issue book controller
- b. Every use case involve several interactions among a group of objects
- c. Each object involved in the execution of a use case plays its part to help complete the execution of the use case
- d. A controller object co-ordinates the activities of a set of collaborating objects to to deliver the results corresponding to an actor request
- e. For every use case, a separate controller object should be created and given the responsibility to handle actor requests.

53. What is the advantage of using a controller class?

- Effectively decouples the boundary and entity objects from one another, reducing design complexity and making the system tolerant to changes of the user interface and processing logic.
- Embody most of the business processing logic required for the realization of the use case

54. What is the role of a controller object?

- A controller object is given the charge of realizing the behavior associated with the use case by making use of the services of the required entity and boundary classes.
- For example, the controller of the renewBook use case in a library automation system may first request to retrieve and supply the exact books that have been borrowed

55. Is exactly one controller classes required per use case?

- Each use case is normally realized using one controller object
- More complex use cases that need to implement significant business logic may require more than one controller object to realize the use case
- For example, the use cases require the controller object to transit through the number of states

56. What if some classes are missed in the domain model?

- For creating a domain model, create a rough conceptual model where the emphasis is on finding the obvious concepts expressed in the requirements while deferring a detailed investigation
- Later during the design process, the conceptual model is incrementally refined and extended
- In this process, some classes that were missed in the class model would be identified and added, and some of the classes may be split into multiple classes, and even some classes not considered necessary may be deleted.
- The methods and the attributes of the classes as well as the class relationships would be established and added later on.

57. Classify object identification techniques

- a. Grammatical analysis of the problem description
- b. Derivation from data flow
- c. Derivation from the entity relationship(E-R) diagram

58. What is Booch's Object Identification Method

- Booch's object identification approach requires a processing narrative of the given problem to be first developed.
- The processing narrative describes the problem and discusses how it can be solved.
- The objects are identified by noting down the nouns in the processing narrative. Synonym of a noun must be eliminated.
- If an object is required to implement a solution, then it is said to be part of the solution space.
- Otherwise, if an object is necessary only to describe the problem, then it is said to be a part of the problem space.

59. What are the criteria used to eliminate nouns after lexical analysis?

- Users
- True name word
- Retained information
- Multiple attributes
- Common operations

60. List out any two OOD Goodness criteria?

Coupling guidelines. The number of messages between two objects or among a group of objects should be minimum. Excessive coupling between objects is determined to modular design and prevents reuse.

Cohesion guideline. In OOD, cohesion is about three levels

- **Cohesiveness of the individual methods** .Cohesiveness of each of the individual method is desirable, since it assumes that each method does only a well-defined function.
- **Cohesiveness of the data and methods within a class.** This is desirable since it assures that the methods of an object do actions for which the object is naturally responsible, i.e. it assures that no action has been improperly mapped to an object.
- **Cohesiveness of an entire class hierarchy.** Cohesiveness of methods within a class is desirable since it promotes encapsulation of the objects.

61. Define collaboration diagrams. (Nov 2017)

A collaboration diagram, also called a communication diagram or interaction diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML).

62. Differentiate activity diagram with state chart diagrams. (Nov 2017)

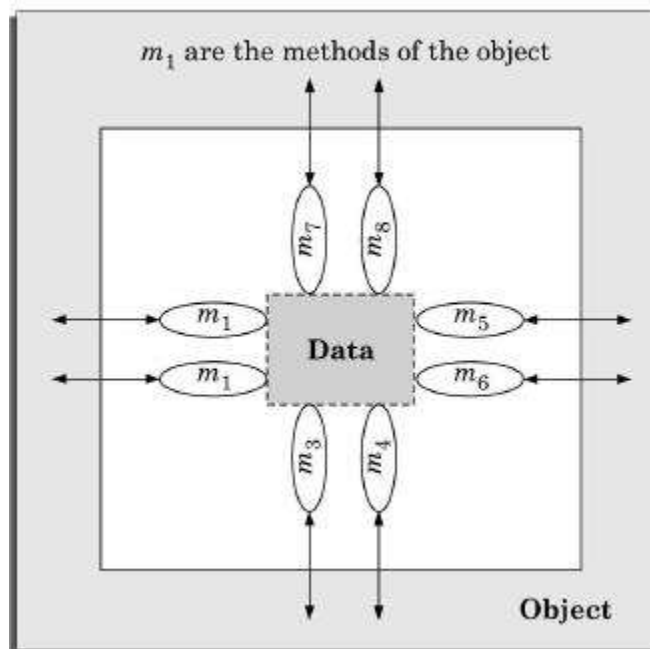
Activity diagrams are similar to the procedural flow charts. The difference is that activity diagrams support description of parallel activities and synchronization aspects involved in different activities. **A state chart diagram** is normally used to model how the state of an object changes in its lifetime.

11 Marks

1. Explain Object-Oriented Concepts

Objects:

- A system is designed as a set of interacting objects
- A tangible real-world entity such as library member, an employee, or a book etc.,
- Certain conceptual entities as objects as well as to simplify the solution eg. A scheduler, a controller, etc.
- When the system is analyzed, developed, and implemented in terms of natural objects, it becomes easy to understand the design and the implementation of the system
- Provide an excellent decomposition of a large problem into small parts
- Consists of some data that is private to the object and a set of functions that operate on those data.
- The functions of an object have the sole authority to operate on the private data of that object.
- Each object can be thought of as hiding its internal data from the other objects.
- Data Hiding(Data abstraction)
 - A mechanism of hiding data from other objects



- Each object stores some data and supports certain operations on the stored data
- For eg, LibraryMember object of a library automation application. The private data of a LibraryMember object can be:
 - Name of the member
 - Membership number
 - Address
 - Phone number
 - E-mail number
 - Date when admitted as a member
 - Membership expiry date
 - Books outstanding, etc
- The operations supported by a LibraryMember object can be:
 - Issue-book
 - Find-books-outstanding
 - Find-books-overdue
 - Return-book
 - Find-membership-details, etc
- The data stored internally in an object are called its **attributes**, and the functions supported by an object are called its **methods**.
- Hybrid application
 - An application, which consists of objects as well as certain primitive data that are that are not objects. Eg. Int, float, etc.,
- Smalltalk
 - First object-oriented programming language
 - Developed in the 1970s at the Xerox research center
 - Pure object-oriented language – language consisted of objects only.
- Objects are passed as reference arguments to methods, whereas primitive data types are passes by value.

Class:

- Similar objects constitute a class.
- Objects possessing similar attributes and having similar methods constitute a class.
- For eg. The set of all library members constitute a class in a library automation application. Each library member object has attributes such as issue-book, return-book, etc
- It serves as a template for object creation
 - An abstract data type, which has the following The data of an object can be accessed only through its methods.
 - We can instantiate a class into objects that is a class is a type.

Methods

- The operations (such as create, issue, return etc.) supported by an object are implemented in the form of methods. The terms operation and method are often used interchangeably.
- An operation is a specific responsibility of a class. A method is an implementation of the responsibility.
- The implementation of a class responsibility through multiple methods is known as method overloading. For example, suppose one of the responsibility of a class named Circle is to create instances of itself. Assume that the class provides three definitions for the create operation: `int create()`, `int create(int radius)` and `int create(float x, float y, int radius)`. In this case, we say that create is an overloaded method.
- Methods are the only means available to other objects for accessing and manipulating the data of another object.
- The set of valid messages to an object constitutes its protocol.

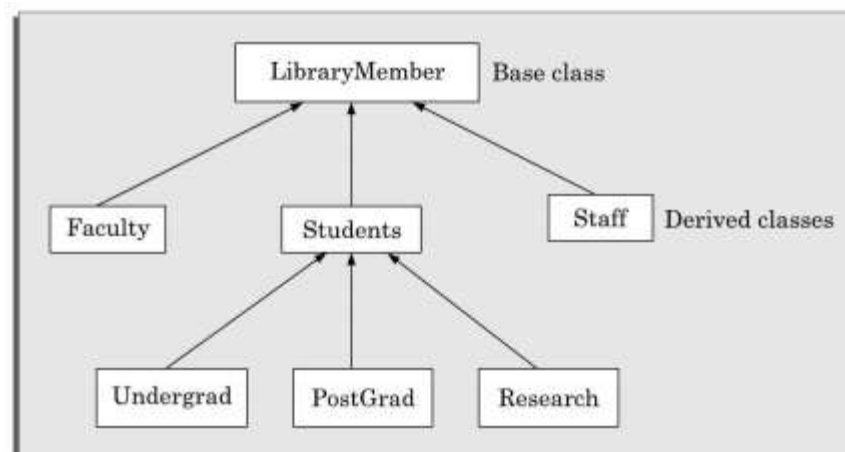
Class relationships

Classes can be related to each other in the following four ways

- Inheritance
- Association
- Aggregation and composition
- Dependency

Inheritance

- The inheritance is used to define a new class by extending the features of modifying an existing class. The original class is called the base class(super class or parent class) and the new class obtained through inheritance is called derived class(sub class or child class)
- An example of inheritance has been shown in figure which shows the classes Faculty, Students, and Staff as having been derived from the base class LibraryMember through an inheritance relationship.



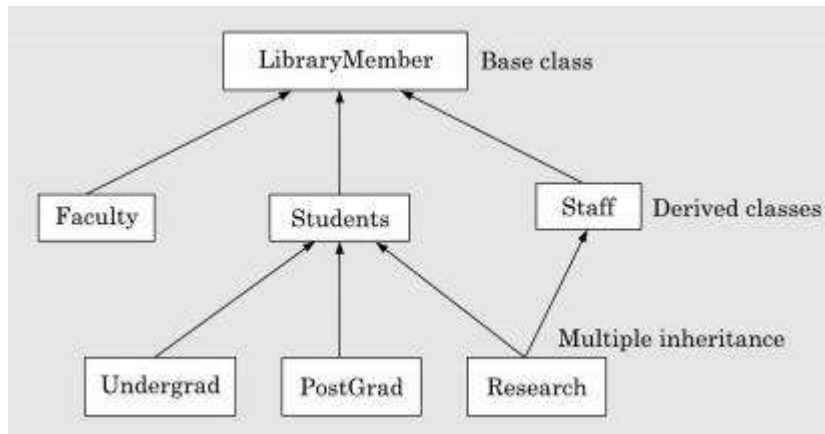
Library information system

- Each derived class can be considered as a specialization of its base class because it modifies or extends the basic properties of the base class in certain ways.
- When a new definition of a method that existed in the base class is provided in a derived class, the method is said to be **overridden** in the derived class.

- Object-based languages not support inheritance mechanism.
- The important advantages of using the inheritance mechanism in programming include **code reuse and simplicity** of program design

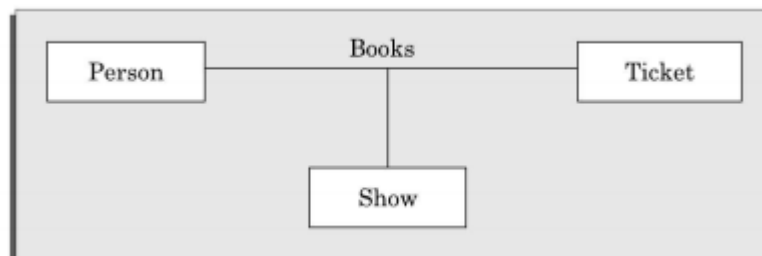
Multiple inheritances

Multiple inheritance is a mechanism by which a subclass can inherit attributes and methods from more than one base class.



Association and link

- If one class is associated with another, then the corresponding objects of the two classes know each other. That is, each object of one class in the association relationship knows the address of the corresponding object of the other class.
- Example : Library Member *borrow*s Books, *borrow*s is the association between the class LibraryMember and the class Book.
- When two classes are associated, the relationship between two objects of the two corresponding classes is called a link.
- An Association describes a group of similar links. A link can be considered as an instance of an association relation.
- A link can be considered to be a tuple. Consider the following example. Suppose Amit has borrowed the book Graph Theory. Borrowed is the link between the objects Amit and the Graph Theory book. This link can also be expressed as the ordered list of object instances { Amit, Graph Theory}.
- Binary association – an association exists between two classes
- Ternary association – an association exists among three or more different classes. Eg. A person books a ticket for a certain show. Association exists between the classes Person, Ticket, and show.



Ternary association

- Recursive association – Two (or more) different objects of the same class are linked by the association relationship.

Composition and aggregation

- Composition and aggregation represent part/whole relationships among different objects.
- Objects which contain other objects are called composite objects.
- The composition/aggregation relationship is also known as a has relationship. Example, studentSemesterRegistration has many course objects Composition can occur in a hierarchy of levels.
- The composition and aggregation relationships are not reflexive. The association relation cannot be circularly defined. That is, an object cannot contain object of the same type as itself.



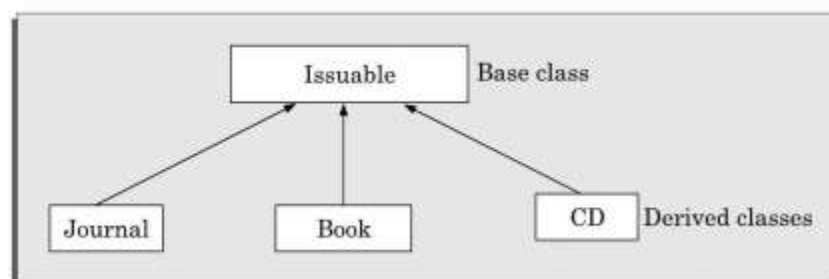
Example of aggregation relationship

Dependency

- A dependency relation is a form of association relation between two classes.
- A dependency relation between two classes shows that any change made to the independent class would require the corresponding change to be made to the dependent class.
- Two important reasons for dependency relation to exist between two classes are the following:
 - A class invokes the methods provided by another class
 - A class implements an interface class. If the properties of the interface class are changed, then a change becomes necessary to the class implementing the interface class as well.

Abstract class

- Classes that are not intended to produce instances of themselves are called abstract classes. Abstract classes cannot be instantiated.
- Abstract classes merely exist so that behavior common to a variety of classes can be factored into one common location, where they can be defined once
- By using abstract classes, code reuse can be enhanced and the effort required to develop software brought down.
- Abstract classes support generic methods, and the subclasses of the abstract classes are expected to provide specific implementations of these methods.



An example of abstract class

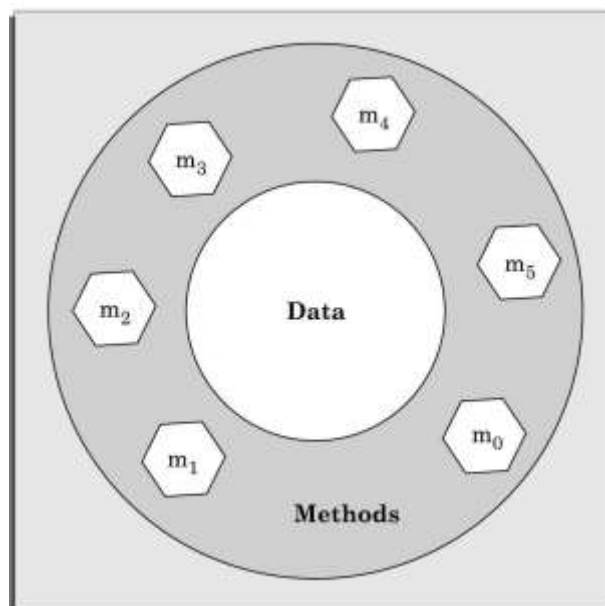
Key concepts:

Abstraction

- The abstraction mechanism allows us to represent a problem in a simpler way by considering only those aspects that are relevant to some purpose and omitting all other details that are irrelevant.
- Abstraction is supported in two different ways in an object-oriented designs(OODs) as follows:
 1. Feature abstraction:
 2. Data abstraction.
- **Feature abstraction** – Every class is a simplified representation of its derived classes and retains only those features that are common to all its children classes and ignores the rest of the features. Inheritance provides feature abstraction.
- **Data abstraction** – Each object hides from other objects the exact way in which it stored its internal information is organized and manipulated. For example, a stack object might store its internal data either in the form of an array of values or in the form of a linked list. Other objects would not know how exactly this object has stored its data and how it internally manipulates its data.
- **Advantage of data abstraction** – It reduces coupling among various objects, Therefore, it leads to a reduction of the overall complexity of a design, and helps in easy maintenance and code reuse.

Encapsulation

- The data of an object is encapsulated within its methods.



Encapsulation Advantages:

- Protection from unauthorized data access – Encapsulation protects an object's variables from corruption by other objects.
- Data hiding – Encapsulation hides the internal structure of an object so that all interactions with the object are simple and standardized.
- Weak coupling – Since objects communicate among each other using messages only, they are weakly coupled. Weak coupling among objects enhances understandability of design since each object can be studied and understood in isolation from other objects.

Polymorphism

Polymorphism literally means poly (many) morphism (forms). It denotes that a method may respond (behave) differently even when the same operation is initiated depending on the exact polymorphic method that is bound.

Polymorphism can be implemented using the following two mechanisms:

- Static polymorphism
- Dynamic polymorphism

Static polymorphism(static binding):

- It occurs when multiple methods with same operation name exist.
- Eg. A class named Circle has three definitions for the create operation: int create(), int create(int radius), and int create(float x, float y, int radius).

```
class Circle{
    private float x, y, radius;
    private int fillType;

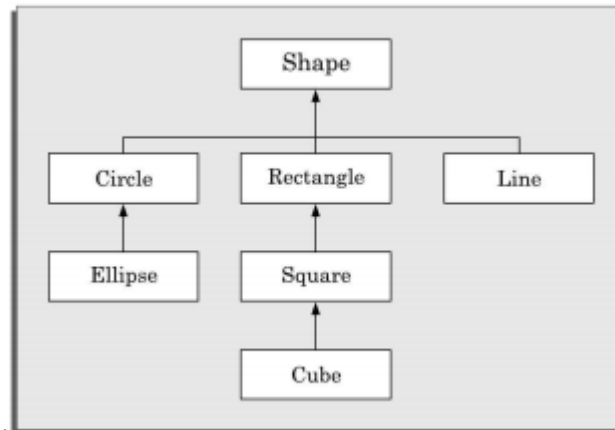
    public create();
    public create(float x, float y, float radius);
    public create(float x, float y, float radius, int fillType);
}
```

- When multiple methods implement the same operation, then the mechanism used is called method overloading.

Dynamic polymorphism (Dynamic binding):

- It occurs when some methods of the base class are overridden by the derived classes and the instances of the derived classes are stored in instances of the base class.
- Even when the method of an object of the base class is invoked, an appropriate overridden method of a derived class would be invoked depending on the exact object that may have been assigned at the run-time to the object of the base class.

- The main advantage of dynamic binding is that it leads to elegant programming and facilitates code reuse and maintenance



Class hierarchy of geometric objects

Related Technical Terms

- Persistence
- Agents
- Widgets

Persistence

Objects usually get destroyed once a program finishes execution. Persistent objects are stored permanently. That is, they live across different executions. An object can be made persistent by maintaining copies of the object in a secondary storage or in a database.

Agents(active object):

- A passive object is one that performs some action only when requested through invocation of some of its methods. An agent monitors events occurring in the application and takes actions autonomously.

Widgets(window object):

- A widget is a primitive object used for graphical user interface (GUI) design. More complex graphical user interface design primitives can be derived from the basic widget using the inheritance mechanism.
- A widget maintains internal data such as the geometry of the window, background and foreground colours of the window, cursor shape and size, etc.

Advantage of OOD:

- Code and design reuse
- Increased productivity

- Ease of testing and maintenance
- Better code and design understandability

2. Explain UML and Use Case Model

Unified Modeling Language (UML)

- UML, as the name implies, is a modeling language. It may be used to visualize, specify, construct, and document the artifacts of a software system.
- It provides a set of notations (e.g. rectangles, lines, ellipses, etc.) to create a visual model of the system. Like any other language, UML has its own syntax (symbols and sentence formation rules) and semantics (meanings of symbols and sentences).
- UML is not a system design or development methodology, but can be used to document object-oriented and analysis results obtained using some methodology.

Origin of UML

- In the late 1980s and early 1990s, there was a proliferation of object-oriented design techniques and notations.
- Different software development houses were using different notations to document their object-oriented designs.
- These diverse notations used to give rise to a lot of confusion.
- UML was developed to standardize the large number of object-oriented modeling notations that existed and were used extensively in the early 1990s. The principles ones in use were:
 - Object Management Technology [Rumbaugh 1991]
 - Booch's methodology [Booch 1991]
 - Object-Oriented Software Engineering [Jacobson 1992]
 - Odell's methodology [Odell 1992]
 - Shaler and Mellor methodology [Shaler 1992]
- It is needless to say that UML has borrowed many concepts from these modeling techniques.
- Especially, concepts from the first three methodologies have been heavily drawn upon. UML was adopted by Object Management Group (OMG) as a *de facto* standard in 1997.
- OMG is an association of industries which tries to facilitate early formation of standards.
- UML contains an extensive set of notations and suggests construction of many types of diagrams.
- It has successfully been used to model both large and small problems. The elegance of UML, its adoption byOMG, and a strong industry backing have helped UML find widespread acceptance. UML is now being used in a large number of software development projects worldwide.

Model

- A model captures aspects important for some application while omitting (or abstracting) the rest.
- A model in the context of software development can be graphical, textual, mathematical, or program code-based. Models are very useful in documenting the design and analysis results.
- Models also facilitate the analysis and design procedures themselves. Graphical models are very popular because they are easy to understand and construct. UML is primarily a graphical modeling tool. However, it often requires text explanations to accompany the graphical models.

Need for a model

- An important reason behind constructing a model is that it helps manage complexity. Once models of a system have been constructed, these can be used for a variety of purposes during software development, including the following:
 - Analysis
 - Specification
 - Code generation
 - Design
 - Visualize and understand the problem and the working of a system
 - Testing, etc.
- In all these applications, the UML models can not only be used to document the results but also to arrive at the results themselves. Since a model can be used for a variety of purposes, it is reasonable to expect that the model would vary depending on the purpose for which it is being constructed.
- For example, a model developed for initial analysis and specification should be very different from the one used for design.
- A model that is being used for analysis and specification would not show any of the design decisions that would be made later on during the design stage. On the other hand, a model used for design purposes should capture all the design decisions.
- Therefore, it is a good idea to explicitly mention the purpose for which a model has been developed, along with the model.

UML diagrams

- UML can be used to construct nine different types of diagrams to capture five different views of a system.
- Just as a building can be modeled from several views (or perspectives) such as ventilation perspective, electrical perspective, lighting perspective, heating perspective, etc.; the different UML diagrams provide different perspectives of the software system to be developed and facilitate a comprehensive understanding of the system.
- Such models can be refined to get the actual implementation of the system.
- The UML diagrams can capture the following five views of a system:
 - User's view
 - Structural view
 - Behavioral view
 - Implementation view
 - Environmental view

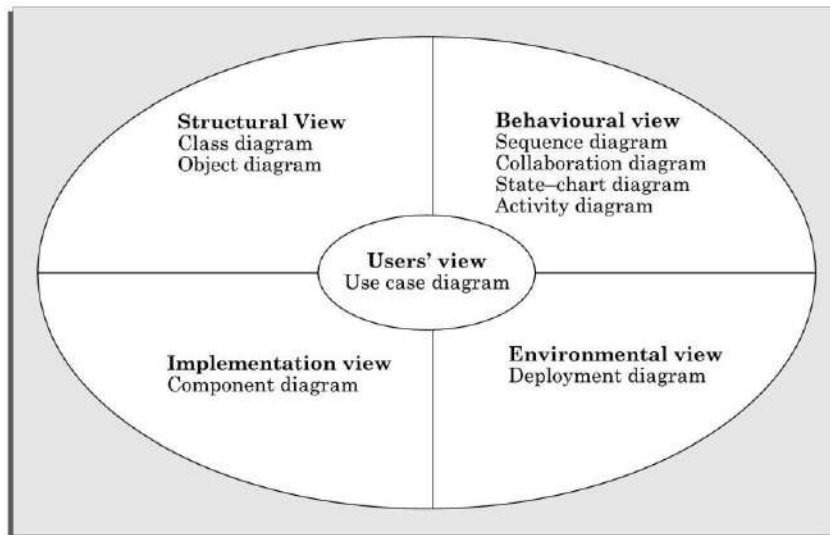


Fig: Different types of diagrams and views supported in UML

User's view:

- This view defines the functionalities (facilities) made available by the system to its users.
- The users' view captures the external users' view of the system in terms of the functionalities offered by the system.
- The users' view is a black-box view of the system where the internal structure, the dynamic behavior of different system components, the implementation etc. are not visible.
- The users' view is very different from all other views in the sense that it is a functional model compared to the object model of all other views. The users' view can be considered as the central view and all other views are expected to conform to this view. This thinking is in fact the crux of any user centric development style.

Structural view:

- The structural view defines the kinds of objects (classes) important to the understanding of the working of a system and to its implementation. It also captures the relationships among the classes (objects).
- The structural model is also called the static model, since the structure of a system does not change with time.

Behavioral view: The behavioral view captures how objects interact with each other to realize the system behavior. The system behavior captures the time-dependent (dynamic) behavior of the system.

Implementation view: This view captures the important components of the system and their dependencies.

Environmental view: This view models how the different components are implemented on different pieces of hardware.

USE CASE MODEL:

- The use case model for any system consists of a set of "use cases".
- Intuitively, use cases represent the different ways in which a system can be used by the users.
- A simple way to find all the use cases of a system is to ask the question: "What the users can do using the system?"
- Thus for the Library Information System (LIS), the use cases could be:
 - issue-book

- query-book
- return-book
- create-member
- add-book, etc
- Use cases correspond to the high-level functional requirements.
- The use cases partition the system behavior into transactions, such that each transaction performs some useful action from the user's point of view.
- To complete each transaction may involve either a single message or multiple message exchanges between the user and the system to complete.

Purpose of use cases

- The purpose of a use case is to define a piece of coherent behavior without revealing the internal structure of the system.
- The use cases do not mention any specific algorithm to be used or the internal data representation, internal structure of the software, etc.
- A use case typically represents a sequence of interactions between the user and the system. These interactions consist of one mainline sequence.
- The mainline sequence represents the normal interaction between a user and the system. The mainline sequence is the most occurring sequence of interaction.
- For example, the mainline sequence of the withdraw cash use case supported by a bank ATM drawn, complete the transaction, and get the amount.
- Several variations to the main line sequence may also exist.
- A variation from the mainline sequence occurs when some specific conditions hold.
- For the bank ATM example, variations or alternate scenarios may occur, if the password is invalid or the amount to be withdrawn exceeds the amount balance. The variations are also called alternative paths.
- A use case can be viewed as a set of related scenarios tied together by a common goal. The mainline sequence and each of the variations are called scenarios or instances of the use case. Each scenario is a single path of user events and system activity through the use case.

Representation of use cases

- Use cases can be represented by drawing a use case diagram and writing an accompanying text elaborating the drawing.
- In the use case diagram, each use case is represented by an ellipse with the name of the use case written inside the ellipse.
- All the ellipses (i.e. use cases) of a system are enclosed within a rectangle which represents the system boundary.
- The name of the system being modeled (such as Library Information System) appears inside the rectangle.
- The different users of the system are represented by using the stick person icon. Each stick person icon is normally referred to as an actor.
- An actor is a role played by a user with respect to the system use. It is possible that the same user may play the role of multiple actors. Each actor can participate in one or more use cases.
- The line connecting the actor and the use case is called the communication relationship.
- It indicates that the actor makes use of the functionality provided by the use case. Both the human users and the external systems can be represented by stick person icons.
- When a stick person icon represents an external system, it is annotated by the stereotype <<external system>>.

Example 1:

The use case model for the Tic-tac-toe problem is shown in fig. 7.2. This software has only one use case “play move”. Note that the use case “get-user-move” is not used here. The name “get-user-move” would be inappropriate because the use cases should be named from the users’ perspective.

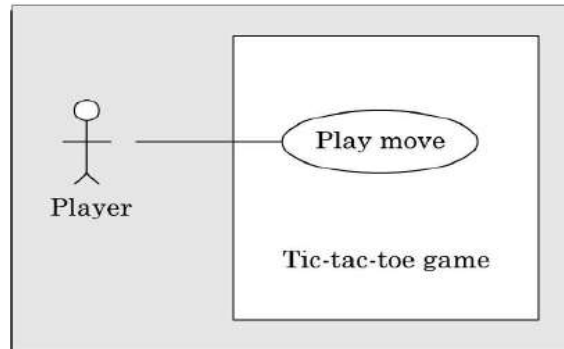


Fig: Use case model for tic-tac-toe game

Text Description

- Each ellipse on the use case diagram should be accompanied by a text description.
- The text description should define the details of the interaction between the user and the computer and other aspects of the use case.
- It should include all the behavior associated with the use case in terms of the mainline sequence, different variations to the normal behavior, the system responses associated with the use case, the exceptional conditions that may occur in the behavior, etc.
- The behavior description is often written in a conversational style describing the interactions between the actor and the system.
- The text description may be informal, but some structuring is recommended. The following are some of the information which may be included in a use case text description in addition to the mainline sequence, and the alternative scenarios.

Contact persons:

- This section lists the personnel of the client organization with whom the use case was discussed, date and time of the meeting, etc.

Actors:

- In addition to identifying the actors, some information about actors using this use case which may help the implementation of the use case may be recorded.

Pre-condition:

- The preconditions would describe the state of the system before the use case execution starts.

Post-condition:

- This captures the state of the system after the use case has successfully completed.

Non-functional requirements: This could contain the important constraints for the design and implementation, such as platform and environment conditions, qualitative statements, response time requirements, etc.

Exceptions, error situations: This contains only the domain-related errors such as lack of user’s access rights, invalid entry in the input fields, etc. Obviously, errors that are not domain related, such as software errors, need not be discussed here.

Sample dialogs: These serve as examples illustrating the use case.

Specific user interface requirements: These contain specific requirements for the user interface of the use case. For example, it may contain forms to be used, screen shots, interaction style, etc.

Document references: This part contains references to specific domain-related documents which may be useful to understand the system operation.

Example 2:

The use case model for the Supermarket Prize Scheme is shown in fig. As discussed earlier, the use cases correspond to the high-level functional requirements. From the problem description and the context diagram in fig., we can identify three use cases: “register-customer”, “register-sales”, and “select-winners”. As a sample, the text description for the use case “register-customer” is shown.

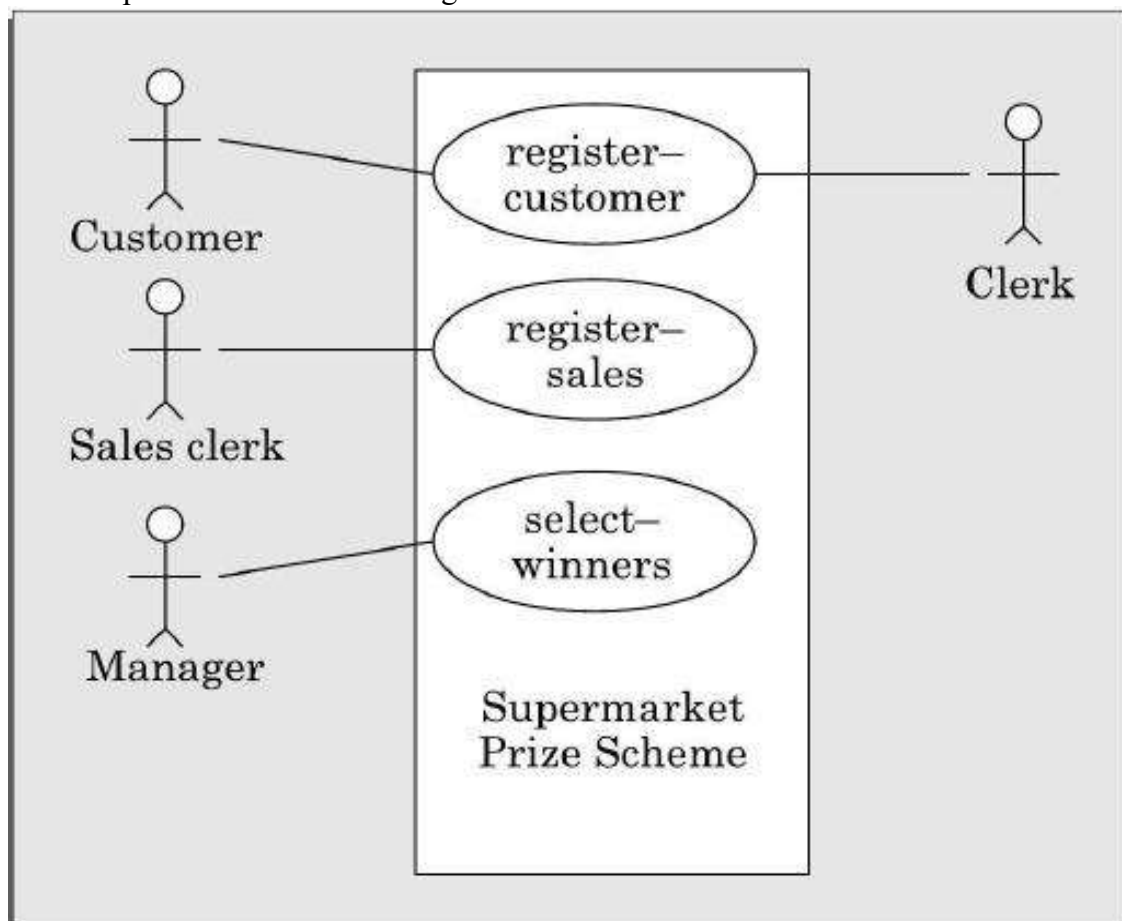


Fig:Use case model for Supermarket Prize Scheme

Text description

U1: register-customer: Using this use case, the customer can register himself by providing the necessary details.

Scenario 1: Mainline sequence

1. **Customer:** select register customer option.
2. **System:** display prompt to enter name, address, and telephone number.

3. Customer: enter the necessary values.

4. System: display the generated id and the message that the customer has been successfully registered.

Scenario 2: at step 4 of mainline sequence

- **System: displays the message that the customer has already registered.**

Scenario 2: at step 4 of mainline sequence

1. System: displays the message that some input information has not been entered. The system display a prompt to enter the missing value.

The description for other use cases is written in a similar fashion.

Utility of use case diagrams

- From use case diagram, it is obvious that the utility of the use cases are represented by ellipses.
- They along with the accompanying text description serve as a type of requirements specification of the system and form the core model to which all other models must conform. But, what about the actors (stick person icons)?
- One possible use of identifying the different types of users(actors) is in identifying and implementing a security mechanism through a login system, so that each actor can involve only those functionalities to which he is entitled to.
- Another possible use is in preparing the documentation (e.g. users' manual) targeted at each category of user. Further, actors help in identifying the use cases and understanding the exact functioning of the system.

Factoring of use cases

- It is often desirable to factor use cases into component use cases. Actually, factoring of use cases are required under two situations.
- First, complex use cases need to be factored into simpler use cases. This would not only make the behavior associated with the use case much more comprehensible, but also make the corresponding interaction diagrams more tractable. Without decomposition, the interaction diagrams for complex use cases may become too large to be accommodated on a single sized (A4) paper.
- Secondly, use cases need to be factored whenever there is common behavior across different use cases. Factoring would make it possible to define such behavior only once and reuse it whenever required.
- It is desirable to factor out common usage such as error handling from a set of use cases.
- This makes analysis of the class design much simpler and elegant. However, a word of caution here.
- Factoring of use cases should not be done except for achieving the above two objectives. From the design point of view, it is not advantageous to break up a use case into many smaller parts just for the sake of it.

UML offers three mechanisms for factoring of use cases as follows:

Generalization

- Use case generalization can be used when one use case that is similar to another, but does something slightly differently or something more.
- Generalization works the same way with use cases as it does with classes. The child use case inherits

the behavior and meaning of the parent use case.

- The notation is the same too (as shown in fig. 7.4). It is important to remember that the base and the derived use cases are separate use cases and should have separate text descriptions.

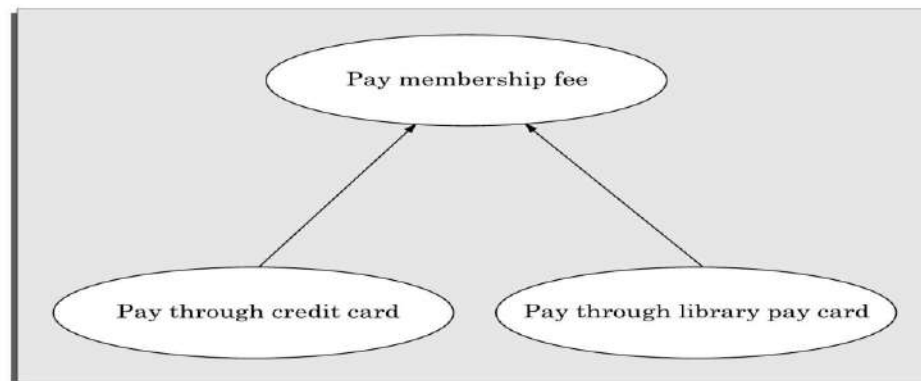
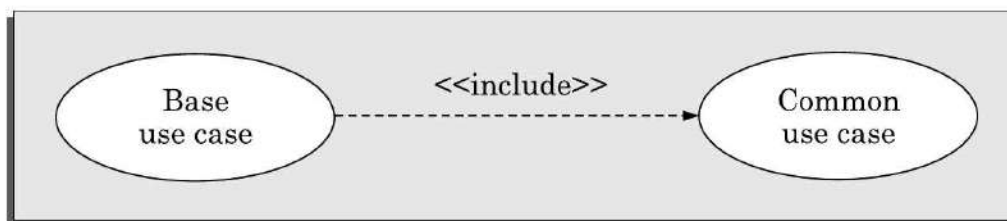


Fig: Representation of use case generalization

Includes

- The includes relationship in the older versions of was known as the uses relationship.
- The includes relationship involves one use case including the behavior of another use case in its sequence of events and actions.
- The includes relationship occurs when a chunk of behavior that is similar across a number of use cases.
- The factoring of such behavior will help in not repeating the specification and implementation across different use cases.
- Thus, the includes relationship explores the issue of reuse by factoring out the commonality across use cases.
- It can also be gainfully employed to decompose a large and complex use cases into more manageable parts. As shown in fig., the includes relationship is represented using a predefined stereotype <<include>>.



lly includes the behavior of k-reservation use case. The her. The common use case provided for it.

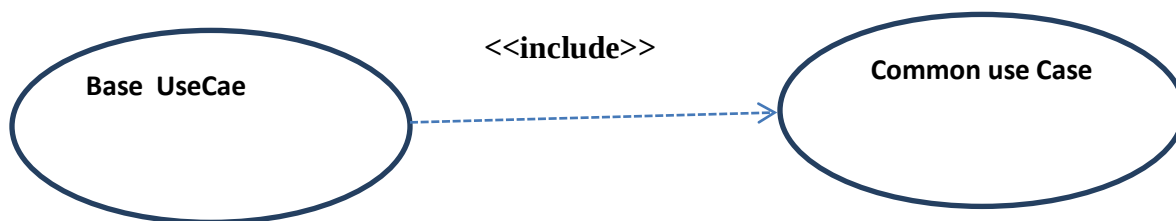
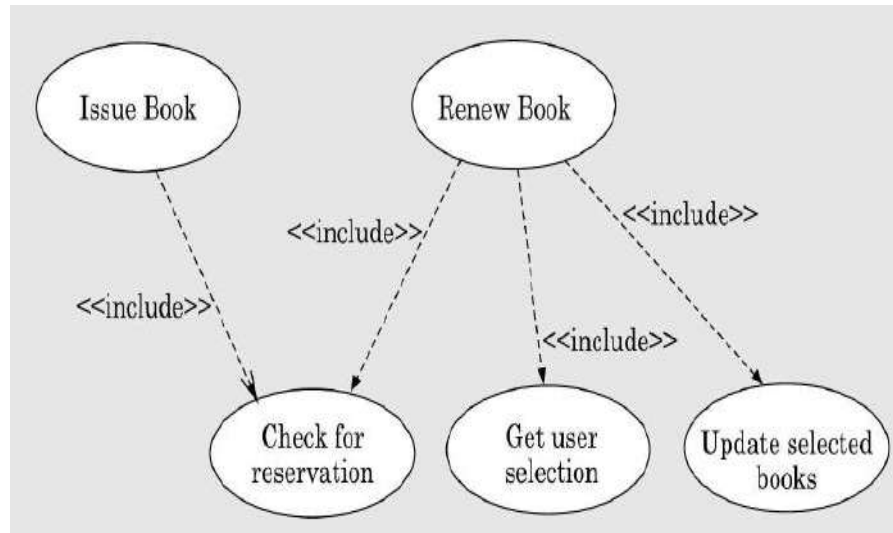


Fig.: Representation of use case inclusions

- The main idea behind the extends relationship among the use cases is that it allows you to show optional system behavior.
- An optional system behavior is extended only under certain conditions. This relationship among use cases is also predefined as a stereotype as shown in fig.
- The extends relationship is similar to generalization. But unlike generalization, the extending use case can add additional behavior only at an extension point only when certain conditions are satisfied.

- The extension points are points within the use case where variation to the mainline (normal) action sequence may occur.

The extends relationship is normally used to capture alternate paths or scenario



Example – use case inclusion

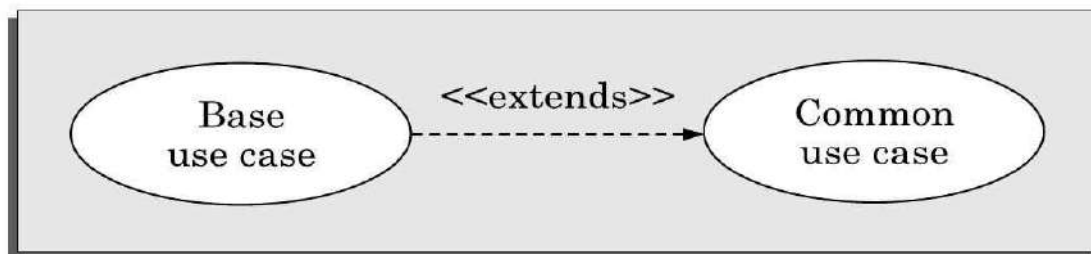


Fig: Example use case extension

Organization of use cases

- When the use cases are factored, they are organized hierarchically.
- The high-level use cases are refined into a set of smaller and more refined use cases as shown in fig.
- Top-level use cases are super-ordinate to the refined use cases. The refined use cases are sub-ordinate to the top-level use cases.
- Note that only the complex use cases should be decomposed and organized in a hierarchy.
- It is not necessary to decompose simple use cases. The functionality of the super-ordinate use cases is traceable to their sub-ordinate use cases.
- Thus, the functionality provided by the super-ordinate use cases is composite of the functionality of the sub-ordinate use cases.
- In the highest level of the use case model, only the fundamental use cases are shown. The focus is on the application context.
- Therefore, this level is also referred to as the context diagram. In the context diagram, the system limits are emphasized.
- In the top-level diagram, only those use cases with which external users of the system.
- The subsystem-level use cases specify the services offered by the subsystems. Any number of levels

involving the subsystems may be utilized.

In the lowest level of the use case hierarchy, the class-level use cases specify the functional fragments or operations offered by the classes.

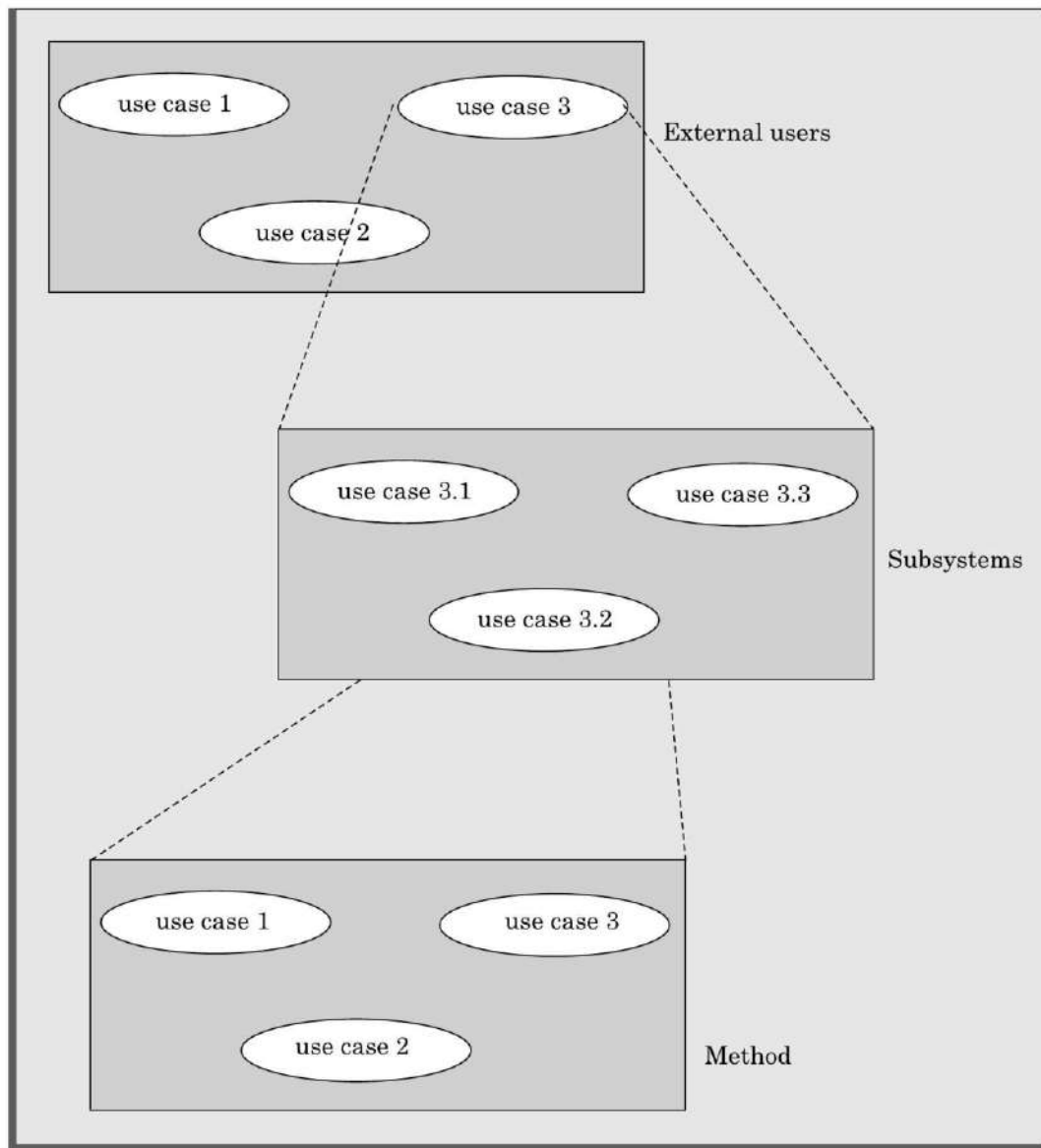


Fig: Hierarchical organization of use cases

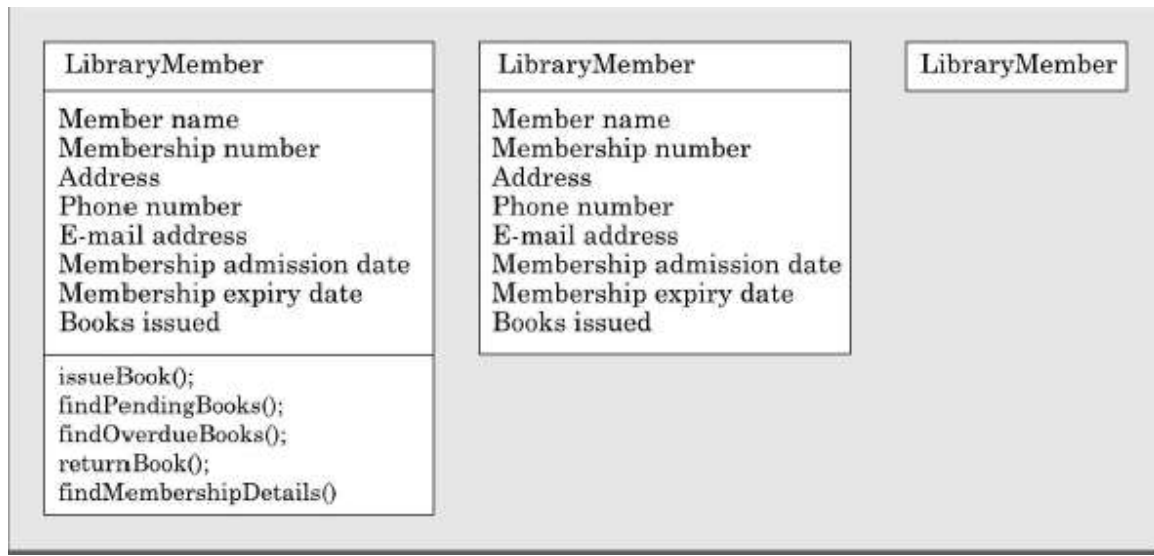
3.Explain Class Diagram

Class Diagram:

- A class diagram describes the static structure of a system.
- Shows how a system is structured rather than how it behaves.
- The static structure of a system comprises of a number of class diagrams and their dependencies.
- Class diagram are classes and their relationships – generalization, aggregation, association and various kinds of dependencies

Classes

- The classes represent entities with common features – attributes and operations.
- Classes are represented as solid outline rectangles with compartments.
- Classes have a mandatory name compartment where the name is written centred in boldface.
- The class name is usually written using mixed case convention and begins with an uppercase. Eg. LibraryMember
- Object names written using a mixed case convention, but starts with a small case letter. Eg. studentMember



Attributes

- An attribute is a named property of a class.
- It represents the kind of data that an object might contain.
- Attributes are listed with their names, and may optionally contain specification of their type, an initial value, and constraints.
- Attribute names may be followed by square brackets containing a multiplicity expression. Eg. sensorStatus[10].
- The multiplicity expression indicates the number of attributes per instance of the class.
- An attribute without square brackets must hold exactly one value.
- The type of an attribute is written by following the attribute name with a colon and the type name. eg. sensorStatus[1]:Int
- The attribute name may be followed by an initialization expression. Eg. sensorStatus[1]:Int=0.

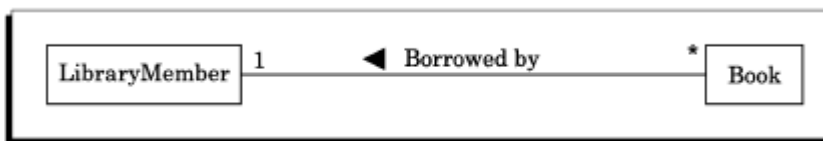
Operation

- The operation names are left justified, in plain type, and always begin with a lower case letter.
- Abstract operations are written in italics.
- The parameters of a function may have a kind specified.
- The kind may be “in” indicates the parameter is passed into the operation and “out” indicates the parameter is only returned from the operation or “inout” indicating that the parameter is used for passing data into the operation and getting result from the operation. The default is “in”

- Static polymorphism – multiple methods implementing same operation
- An operation may have a return type consisting of a single return type expression, eg. `issueBook(in bookName):Boolean`

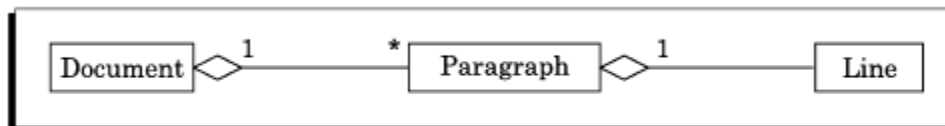
Association

- Associations are needed to enable objects to communicate with each other. An association describes a connection between classes.
- The association relation between two objects is called object connection or link. Links are instances of associations.
- The name of the association is written alongside the association line. An arrowhead may be placed on the association line to indicate the reading direction of the association.
- The arrowhead should not be misunderstood to be indicating the direction of a pointer implementing an association.
- On each side of the association relation, the multiplicity is noted as an individual number or as a value range.
- The multiplicity indicates how many instances of one class are associated with each other. Value ranges of multiplicity are noted by specifying the minimum and maximum value, separated by two dots,
- An asterisk is a wild card and means many (zero or more). The association of fig. should be read as “Many books may be borrowed by a Library Member”. Observe that associations (and links) appear as verbs in the problem statement



Aggregation

- Aggregation is a special type of association where the involved classes represent a whole-part relationship.
- The aggregate takes the responsibility of forwarding messages to the appropriate parts.
- Thus, the aggregate takes the responsibility of delegation and leadership.
- When an instance of one object contains instances of some other objects, then aggregation (or composition) relationship exists between the composite object and the component object.
- Aggregation is represented by the diamond symbol at the composite end of a relationship.

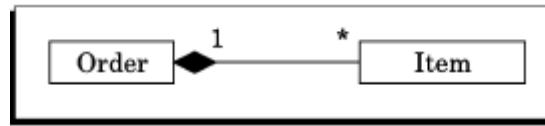


- Aggregation relationship cannot be reflexive (recursive).
- An object cannot contain objects of the same class as itself. Also, the aggregation relation is not symmetric.
- That is, two classes A and B cannot contain instances of each other. However, the aggregation relationship can be transitive. In this case, aggregation may consist of an arbitrary number of levels.

Composition

- Composition is a stricter form of aggregation, in which the parts are existence-dependent on the whole.
- The life of the parts closely ties to the life of the whole. When the whole is created, the parts are created and when the whole is destroyed, the parts are destroyed.

- A typical example of composition is an invoice object with invoice items. As soon as the invoice object is created, all the invoice items in it are created and as soon as the invoice object is destroyed, all invoice items in it are also destroyed.
- The composition relationship is represented as a filled diamond drawn at the composite-end.



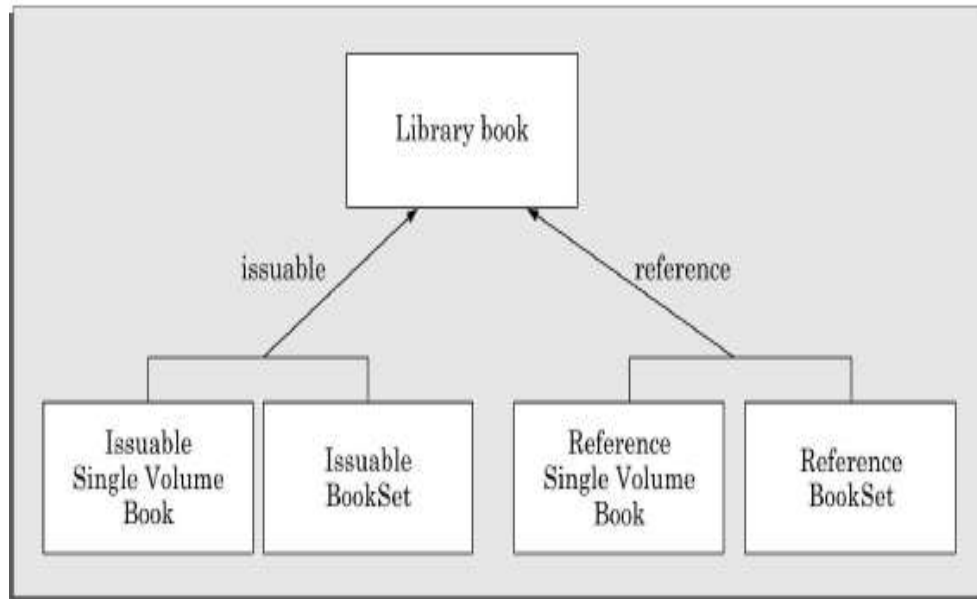
Association vs. Aggregation vs. Composition

- Association is the most general (m:n) relationship. Aggregation is a stronger relationship where one is a part of the other. Composition is even stronger than aggregation, ties the lifecycle of the part and the whole together.
- Association relationship can be reflexive (objects can have relation to itself), but aggregation cannot be reflexive. Moreover, aggregation is anti-symmetric (If B is a part of A, A can not be a part of B).
- Composition has the property of exclusive aggregation i.e. an object can be a part of only one composite at a time. For example, a **Frame** belongs to exactly one **Window** whereas in simple aggregation, a part may be shared by several objects. For example, a **Wall** may be a part of one or more **Room** objects.
- In addition, in composition, the whole has the responsibility for the disposition of all its parts, i.e. for their creation and destruction.
- In general, the lifetime of parts and composite coincides
 - o parts with non-fixed multiplicity may be created after composite itself
 - o parts might be explicitly removed before the death of the composite

For example, when a **Frame** is created, it has to be attached to an enclosing **Window**. Similarly, when the **Window** is destroyed, it must in turn destroy its **Frame** parts

Inheritance

- The inheritance relationship is represented by means of an empty arrow pointing from the subclass to the superclass.
- The arrow drawn from the subclass to the superclass.
- The inheritance arrow from the subclasses combined to a single line when there are many subclasses of a base class.



Dependency

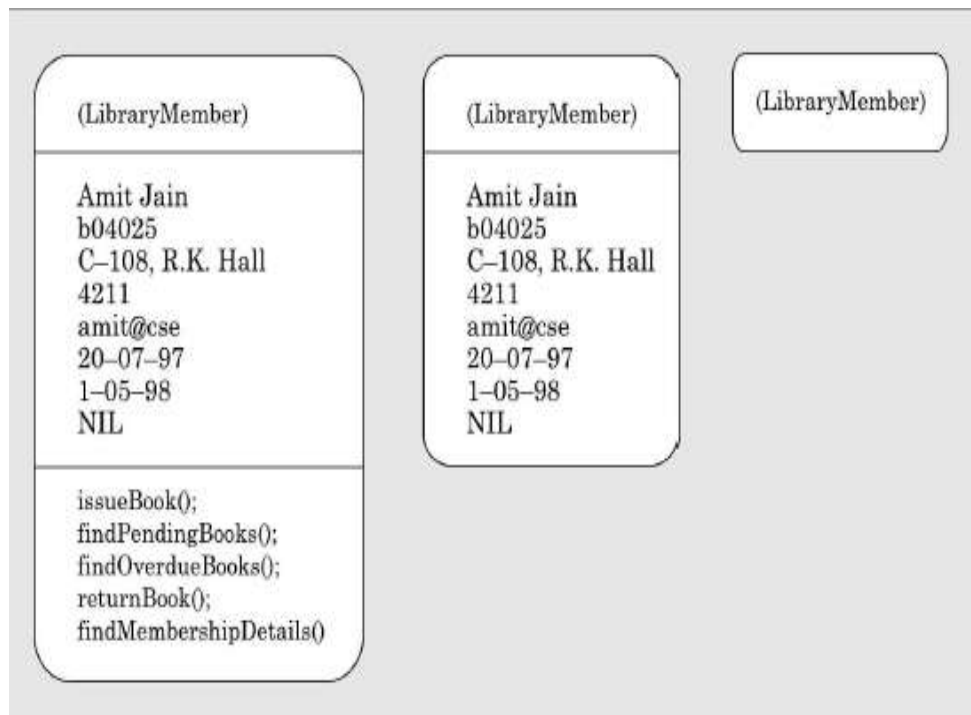
- A dependency relationship is shown as dotted arrow that is drawn from the dependent class to the independent class.

Constraints

- A constraint describes a condition or an integrity rule.
- Constraints are used to describe the permissible set of values of an attribute to specify the pre- and post-conditions for operations, to define certain ordering of items.
- For example, to denote that the books in a library are sorted on ISBN number we can annotate the book class with the constraint {sorted}.
- UML allows you to use any free form expression to describe constraints.
- The only rule is that they are to be enclosed within braces.

Object diagrams (Instance diagram) :

- Object diagrams show the snapshot of the objects in a system at a point in time.
- An object diagram may undergo continuous change as execution proceeds.
- For example, links may get formed between objects and get broken.
- Objects may get created and destroyed, and so on.
- Object diagrams are useful to explain the working of a system



Different types of Objects

4.Explain Interaction Diagram

Interaction Diagrams

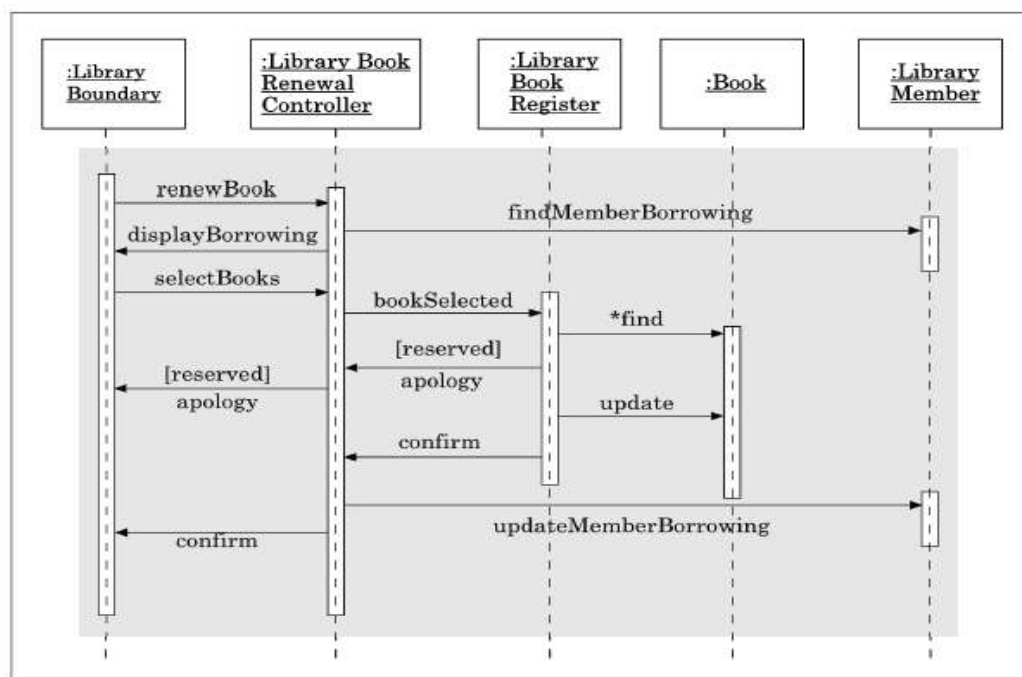
- Interaction diagrams are models that describe how group of objects collaborate to realize some behavior.
- Each interaction diagram realizes the behavior of a single use case.
- An interaction diagram shows a number of example objects and the messages that are passed between the objects within the use case.
- There are two kinds of interaction diagrams:
 - Sequence diagrams and
 - Collaboration diagrams.
- These two diagrams are equivalent in the sense that any one diagram can be derived automatically from the other.
- These two actually portray different perspectives of behavior of the system and different types of inferences can be drawn from them.
- The interaction diagrams can be considered as a major tool in the design methodology.

Sequence Diagram

- A sequence diagram shows interaction among objects as a two dimensional chart.
- The chart is read from top to bottom. The objects participating in the interaction are shown at the top of the chart as boxes attached to a vertical dashed line.
- Inside the box the name of the object is written with a colon separating it from the name of the class and both the name of the object and the class are underlined.
- The objects appearing at the top signify that the object already existed when the use case execution was initiated.
- If some object is created during the execution of the use case and participates in the interaction (a method call), then the object should be shown at the appropriate place on the diagram where it is

created.

- The vertical dashed line is called the object's lifeline. The lifeline indicates the existence of the object at any particular point of time.
- The rectangle drawn on the lifetime is called the activation symbol and indicates that the object is active as long as the rectangle exists.
- Each message is indicated as an arrow between the lifeline of two objects.
- The messages are shown in chronological order from the top to the bottom.
- That is, reading the diagram from the top to the bottom would show the sequence in which the messages occur.
- Each message is labeled with the message name. Some control information can also be included.
- Two types of control information are particularly valuable.
 - A condition indicates that a message is sent, only if the condition is true.
 - An iteration marker shows the message is sent many times to multiple receiver objects as would happen when a collection or the elements of an array are being iterated. The basis of the iteration can also be indicated
- The sequence diagram for the book renewal use case for the Library Automation Software is shown in fig.
- The development of the sequence diagram in the development methodology would help us in determining the responsibilities of the different classes; i.e. what methods should be supported by each class.

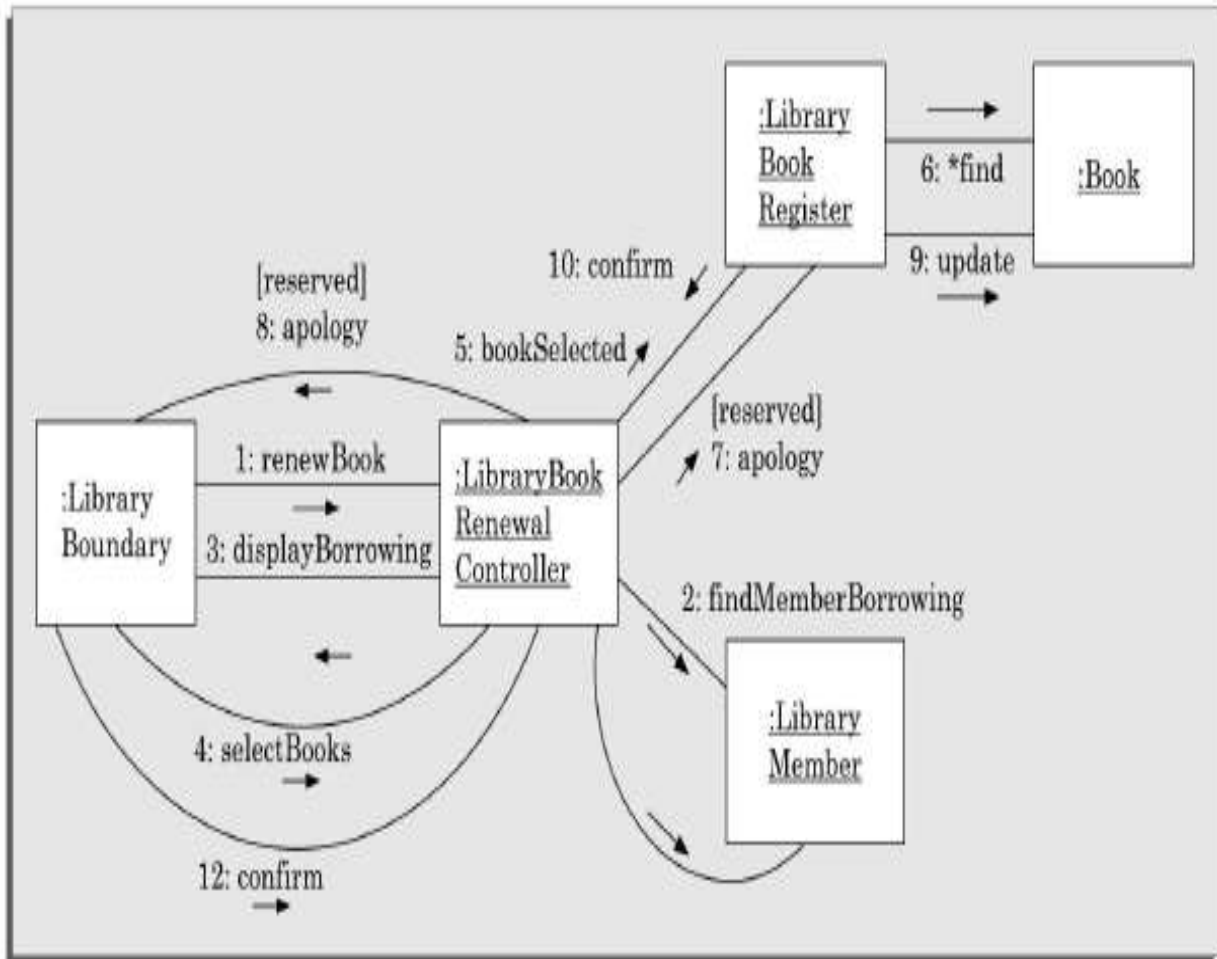


Collaboration Diagram

- A collaboration diagram shows both structural and behavioral aspects explicitly. This is unlike a sequence diagram which shows only the behavioral aspects.
- The structural aspect of a collaboration diagram consists of objects and the links existing between them.
- In this diagram, an object is also called a collaborator. The behavioral aspect is described by the set of messages exchanged among the different collaborators.

- The link between objects is shown as a solid line and can be used to send messages between two objects.
- The message is shown as a labeled arrow placed near the link.
- Messages are prefixed with sequence numbers because they are only way to describe the relative sequencing of the messages in this diagram.
- The collaboration diagram for the example of fig is shown in fig.

The use of the collaboration diagrams in our development process would be to help us to determine which classes are associated with which other classes.



5.Explain Activity diagram and State chart diagram

Activity and State Chart Diagram

Activity diagrams

- The activity diagram is possibly one modeling element which was not present in any of the predecessors of UML.
- No such diagrams were present either in the works of Booch, Jacobson, or Rumbaugh. It is possibly based on the event diagram of Odell [1992] through the notation is very different from that used by Odell.
- The activity diagram focuses on representing activities or chunks of processing which may or may not correspond to the methods of classes.

- An activity is a state with an internal action and one or more outgoing transitions which automatically follow the termination of the internal activity.
- If an activity has more than one outgoing transitions, then these must be identified through conditions.
- An interesting feature of the activity diagrams is the swim lanes. Swim lanes enable you to group activities based on who is performing them, e.g. academic department vs. hostel office.
- Thus swim lanes subdivide activities based on the responsibilities of some components. The activities in a swim lane can be assigned to some model elements, e.g. classes or some component, etc.
- Activity diagrams are normally employed in business process modeling. This is carried out during the initial stages of requirements analysis and specification.
- Activity diagrams can be very useful to understand complex processing activities involving many components.
- Later these diagrams can be used to develop interaction diagrams which help to allocate activities (responsibilities) to classes.
- The student admission process in SMVEC is shown as an activity diagram in fig.. This shows the part played by different components of the Institute in the admission procedure.
- After the fees are received at the account section, parallel activities start at the hostel office, hospital, and the Department.
- After all these activities complete (this synchronization is represented as a horizontal line), the identity card can be issued to a student by the Academic section

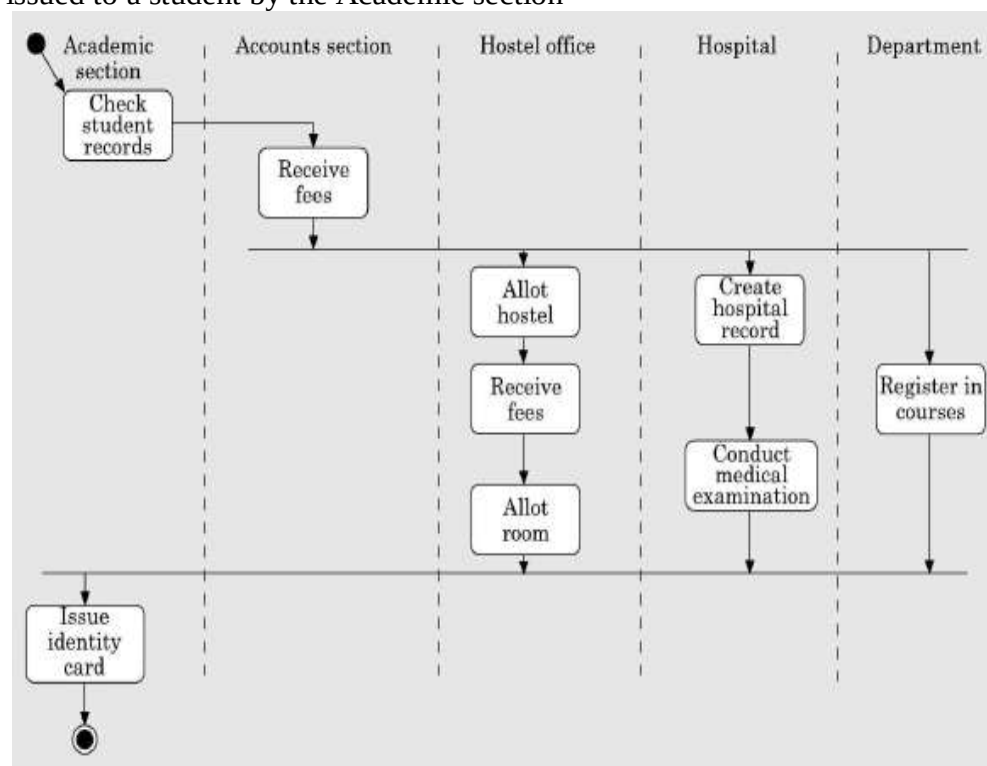


Fig: Activity diagram for student admission procedure at SMVEC

Activity diagrams vs. procedural flow charts

- Activity diagrams are similar to the procedural flow charts.
- The difference is that activity diagrams support description of parallel activities and synchronization aspects involved in different activities.

State chart diagram

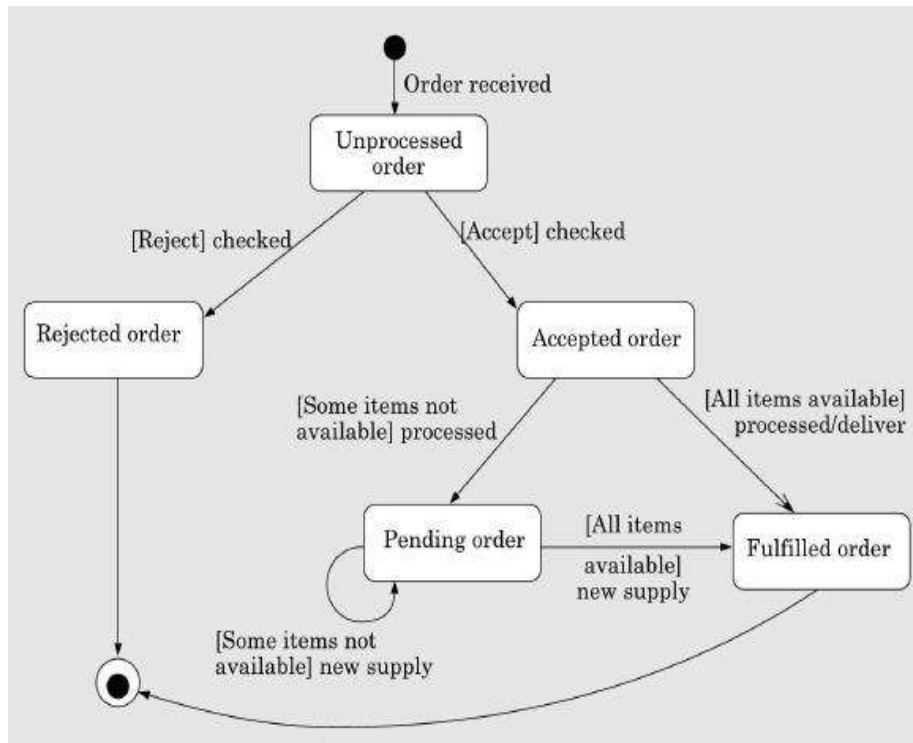
- A state chart diagram is normally used to model how the state of an object changes in its lifetime.
- State chart diagrams are good at describing how the behavior of an object changes across several use

case executions.

- However, if we are interested in modeling some behavior that involves several objects collaborating with each other, state chart diagram is not appropriate.
- State chart diagrams are based on the finite state machine (FSM) formalism.
- An FSM consists of a finite number of states corresponding to those of the object being modeled.
- The object undergoes state changes when specific events occur.
- The FSM formalism existed long before the object-oriented technology and has been used for a wide variety of applications.
- Apart from modeling, it has even been used in theoretical computer science as a generator for regular languages.
- A major disadvantage of the FSM formalism is the state explosion problem.
- The number of states becomes too many and the model too complex when used to model practical systems. This problem is overcome in UML by using state charts.
- The state chart formalism was proposed by David Harel [1990]. A state chart is a hierarchical model of a system and introduces the concept of a composite state (also called nested state).
- Actions are associated with transitions and are considered to be processes that occur quickly and are not interruptible.
- Activities are associated with states and can take longer. An activity can be interrupted by an event.

The basic elements of the state chart diagram are as follows:

- **Initial state.** This is represented as a filled circle.
- **Final state.** This is represented by a filled circle inside a larger circle.
- **State.** These are represented by rectangles with rounded corners.
- **Transition.** A transition is shown as an arrow between two states. Normally, the name of the event which causes the transition is placed along side the arrow. A guard to the transition can also be assigned. A guard is a Boolean logic condition. The transition can take place only if the guard evaluates to true. The syntax for the label of the transition is shown in 3 parts: event[guard]/action.
- An example state chart for the order object of the Trade House Automation software is shown in fig



State chart diagram for an order object

Activity diagram vs. State chart diagram

- Both activity and state chart diagrams model the dynamic behavior of the system. Activity diagram is essentially a flowchart showing flow of control from activity to activity. A state chart diagram shows a state machine emphasizing the flow of control from state to state.
- An activity diagram is a special case of a state chart diagram in which all or most of the states are activity states and all or most of the transitions are triggered by completion of activities in the source state (An activity is an ongoing non-atomic execution within a state machine).
- Activity diagrams may stand alone to visualize, specify, and document the dynamics of a society of objects or they may be used to model the flow of control of an operation. State chart diagrams may be attached to classes, use cases, or entire systems in order to visualize, specify, and document the dynamics of an individual object.

6. Explain Object-Oriented Analysis and Design (OOAD)? (or) explain in detail about object oriented analysis and design methodology. (Nov 2017)

AN OBJECT-ORINETED ANALYSIS AND DESIGN METHODOLOGY

- The results of the analysis activities are be redefined into a design model through several iterations.

The Unified Process

- The unified process is an extensible framework which needs to be customized for specific types of projects.
- The two main characteristics of the unified process are
 - Use case driven
 - Iterative
- Case-driven implies that use cases of the system are considered to be the central and most important view.
- The use case view should be the first one to be constructed and should be refined iteratively into an implementation
- The use case model is the central model. All models that constructed in the subsequent design activities must conform to the use case model

The unified process involves iterating over the following four phases as follows:

1. Inception:

- During this phase, the scope of the project is defined and prototypes may be developed to form a clear idea about the project.

2. Elaboration:

- The functional and the non-functional requirements are captured

3. Construction:

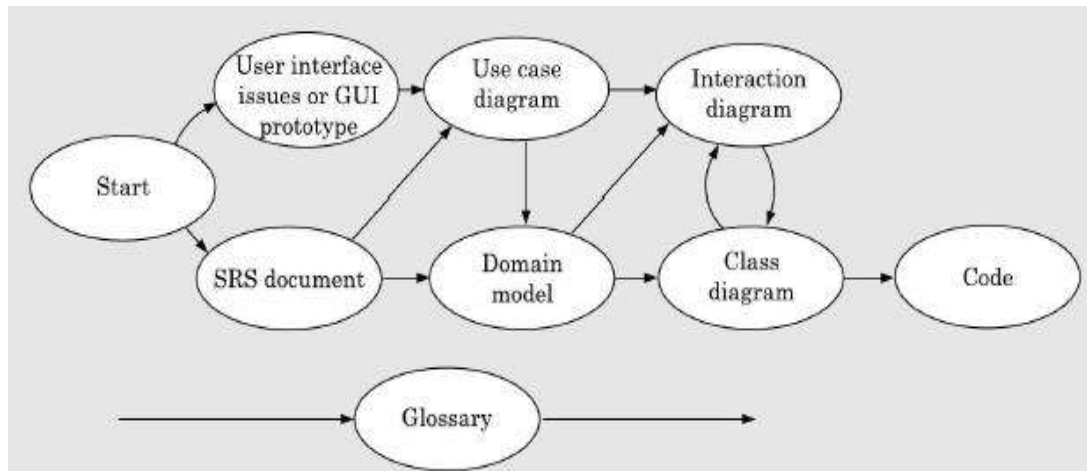
- Analysis, design and implementation activities are carried out.
- Full text descriptions of use cases are written during the construction phase and each use case is taken up for the start of a new iteration.
- System features are implemented in a series of short iterations
- Each iteration results in an executable release of the software

4. Transition:

- The product is installed in the user's environment and maintained.

Overview of the OOAD Methodology

- The use case model is developed first
- Any user-centric development process must conform to the use case model.
- A crucial role in the design process
- The domain model is constructed next through an analysis of the use case model and the SRS document.
- The domain model is refined into a class diagram through a number of iterations involving the interaction diagrams.
- Once the class diagram has been constructed, it can easily be translated to code.



An object-oriented analysis and design process

- A glossary is continually and consciously created and maintained throughout the analysis and design process
- A glossary is a dictionary of terms which can help in understanding the various terms(concepts) used in the constructed model.
- The terms listed in the glossary are essentially concept names
- The glossary lists and defines all the terms that require explanation in order to improve communication and to reduce the risk of misunderstanding

Use Case Model Development

- Captures the high-level user requirements of the system to be developed.
- For every use case, the details all the scenarios of user interactions with the system are captured in an accompanying text description.
- The use cases can be identified from the SRS document.
- The high-level functional requirements correspond to the use cases.
 - It is worth noting that there is a close correspondence with structure of the GUI, the organization of the user's manual into sections, and the packaging of the use cases.
- An overriding principle while identifying and packaging use cases is that there should be a strong correlation between the GUI prototype, the contents of the user's manual and the use case model of the system.
- For example, a text editor software that can be used to create various kinds of text documents.
- The top-level menu options for such a word processing software would be File Operations, Edit, View, Insert, Tools, etc.
- Each of these high-level menu option can be considered to be a container for a set functions. Eg, File menu contain functions such as :load file, store file, save, save as, print, etc
- Each of the menu options in the top-level menu of the GUI would usually correspond to a package in the use case diagram.
- The use case packaging has a strong correlation with the organization of the user's manual into chapters and sections.
- The user's manuals have exactly the same chapters as the first-level packages of the chapter would correspond to the packages in the corresponding first-level package, etc.

- For the case of the word processing software, the different chapters of the user's manual would be arranged into File operations, Edit, View, and Tool operations, etc. corresponding to the GUI menu structure and also the use case packages.
- Carrying out the GUI prototyping in parallel with development of the SRS document is considered an excellent approach
- This involves iterating the presentation aspects of the system with the user.
- After achieving closure on different screens, the corresponding use case model can be developed.
- Even though a close correspondence should exist between the GUI prototype and the use case model, the use cases should not be too tightly tied to the GUI element appearing on the screen, eg. radioButton, pushButton, etc
- The type of the user interface components used may change frequently during software development and more so during later maintenance. However, the functionalities provided by the system do not change so often.
- A use case is most effectively stated from the perspective of the user.
- It should be stated using present tense verb phrases, and should be in active voice, eg. admit patient, issue book, generate report

Common mistakes committed in the use case model development

1. Clutter

- Too many use cases at the top level use case diagram make it very difficult to understand the model
- When large number of use cases are present in the top level of the use case diagram, they should be organized into packages
- Packages are an effective way to manage complexities.
- Each use case package should correspond to one chapter or section of the user's manual or a top-level menu choice in the GUI

2. Too detailed

- Often beginners confuse sub steps of use cases with separate use cases
- For eg, one should may inappropriately define a use case **print receipt**, Whereas it should only be a sub-step of the withdraw cash use case.
- A use case model in which the sub functions of a high-level function are represented as full use cases would make the later analysis and design task difficult

3. Omitting text description

- Omitting text description of the use cases makes it very difficult for any one to gain full understanding of the use case behavior and also makes it very difficult to design the system

4. Overlooking some alternate scenarios

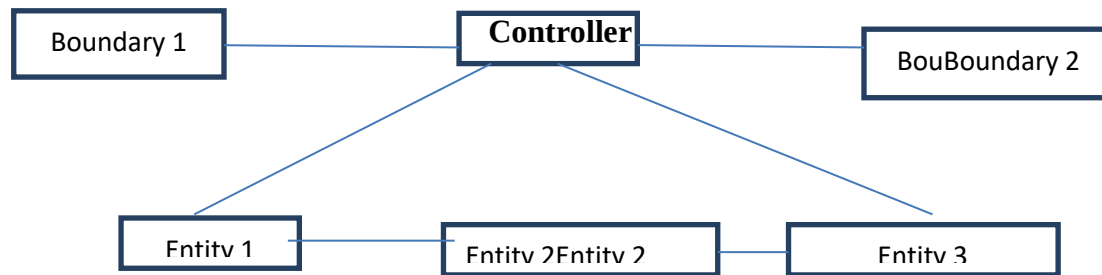
- It is necessary to capture all alternate scenarios of each use case
- Overlooked scenarios later show up as missing functionalities or bugs

Domain Modeling

- Known as conceptual modeling

- A conceptual model depicts the concepts(objects) that are easily identifiable in the problem description
- It contains three types of objects
 - Boundary objects
 - Controller(conceptual) objects
 - Entity objects
- Entity objects
 - objects that correspond to physical entities in the problem description
 - Eg. In a library automation software are book, book register, member register,etc,
 - Hold information such as data tables and files
 - Store data permanently or semi-permanently
 - Store data across use case executions and need to outlive use case executions
 - Carry out only simple processing ontheir stored data such as update, store, search retrieve, etc.
 - Entity objects undergo much less changes during the operation phase of the software compared to the other types of objects and are said to be more stable than the other types of objects.
- Boundary objects(interface objects)
 - objects handle the user interface
 - Eg issue book user interface
 - The boundary objects are those with which the actors interact
 - The boundary objects include screens, menus, forms, dialogs,etc
 - The boundary objects are mainly responsible for evaluating user interactions through suitable graphical user interfaces(GUIs)
 - Responsible to read inputs form the user, validate the inputs, format the outputs, and display the results.
 - The initial identification of the boundary classes can be made by defining one boundary class per actor/use case pair
- Controller objects
 - objects are entirely conceptual
 - Eg. issue book controller
 - Every use case involve several interactions among a group of objects
 - Each object involved in the execution of a use case plays its part to help complete the execution of the use case
 - A controller object co-ordinates the activities of a set of collaborating objects to deliver the results corresponding to an actor request
 - For every use case, a separate controller object should be created and given the responsibility to handle actor requests.
 - For every invocation of a use case by the same (or different) actor, a separate controller object should be instantiated from the corresponding controller class.
 - When an actor invokes a use case, the same controller object should handle all the interactions with that actor.

- To maintain the necessary information about the state of execution of the use case by an actor
- The state information maintained by a controller can be used to identify the out-of-sequence actor request and it can then take appropriate actions.
- A controller object orchestrates the activities of a set of entity classes to implement the behavior of a use case and also manages the user interactions through a set of boundary classes
- Accepts inputs and displays outputs to the users through two boundary classes, it coordinates the activities of the entity classes to realize the behavior associated with the use case.



A typical realization of a use case through the collaboration of boundary, controller, and entity objects

- A domain model is the first-cut class diagram and is obtained from an analysis of the problem description
- In a domain model, no methods or attributes are associated with classes and only the names of the classes are represented.
- While boundary and controller objects can be mechanically identified from an inspection of the use case diagram, identification of entity objects requires practice. So, we can say that the crux of the domain modeling activity is to identify the entity objects judiciously.

The questions which arise in our minds can be as follows:

1. What is the advantage of using a controller class?

- Effectively decouples the boundary and entity objects from one another, reducing design complexity and making the system tolerant to changes of the user interface and processing logic.
- Embodies most of the business processing logic required for the realization of the use case

2. What is the role of a controller object?

- A controller object is given the charge of realizing the behavior associated with the use case by making use of the services of the required entity and boundary classes.
- For example, the controller of the renewBook use case in a library automation system may first request to retrieve and supply the exact books that have been borrowed

3. Is exactly one controller class required per use case?

- Each use case is normally realized using one controller object
- More complex use cases that need to implement significant business logic may require more than one controller object to realize the use case

- For example, the use cases require the controller object to transit through the number of states
- 4. **What if some classes are missed in the domain model?**
 - For creating a domain model, create a rough conceptual model where the emphasis is on finding the obvious concepts expressed in the requirements while deferring a detailed investigation
 - Later during the design process, the conceptual model is incrementally refined and extended
 - In this process, some classes that were missed in the class model would be identified and added, and some of the classes may be split into multiple classes, and even some classes not considered necessary may be deleted.
 - The methods and the attributes of the classes as well as the class relationships would be established and added later on.

Identification of Entity Objects

- The quality of the final design depends on the appropriateness and the completeness of the identified entity objects.
- Several semi-formal and informal approaches proposed for identification of the objects.
- Various object identification techniques classified into the following broad classes
 - Grammatical analysis of the problem description
 - Derivation from data flow
 - Derivation from the entity relationship(E-R) diagram
- The grammatical analysis approach proposed by Grady Booch
- In Booch's approach, the nouns mapped to objects and the verbs mapped to methods.
- Derivation from data flow and the entity relationship diagram used to refine the results obtained using the Booch's object identification methodology.

Booch's Object Identification Method:

Booch's object identification approach requires a processing narrative of the given problem to be first developed.

- The processing narrative describes the problem and discusses how it can be solved.
- The objects are identified by noting down the nouns in the processing narrative. Synonym of a noun must be eliminated.
- If an object is required to implement a solution, then it is said to be part of the solution space.
- Otherwise, if an object is necessary only to describe the problem, then it is said to be a part of the problem space.
- However, several of the nouns may not be objects.
- An imperative procedure name, i.e., noun form of a verb actually represents an action and should not be considered as an object.

A potential object found after lexical analysis is usually considered legitimate, only if it satisfies the following criteria:

Users:

- Appear as name words in the problem description

- The actors and interactions among actors excluded from the entity identification exercise
- Classes corresponding to the names of the actors should be considered that classes sometimes called surrogates.
- For example, in the Library Information System need to store information about each library member such as his name, address, identification number etc.
- This is independent of the fact that the library member also play the role of an actor of to the system

True name word

- An imperative procedure name that is noun form of a verb actually represents an action, and should not be considered as an object.
- For example, **cash withdrawal** in the processing description of an ATM refers to withdrawing cash and is an action and not a noun, and therefore can be deleted from the list of nouns.

Retained information.

- Some information about the object should beremembered for the system to function. If an object does not contain any private data, it can not be expected to play any important role in the system.

Multiple attributes.

- Usually objects have multiple attributes and supportmultiple methods.
- It is very rare to find useful objects which store only a single data element or support only a single method, because an object having only a single data element or method is usually implemented as a part of another object.

Common operations.

- A set of operations can be defined for potentialobjects. If these operations apply to all occurrences of the object, then a class can be defined.
- An attribute or operation defined for a class must apply to each instance of the class.
- If some of the attributes or operations apply only to some specific instances of the class, then one or more subclasses can be needed for these special objects.
- **Example:** Tic-tac-toe

Let us identify the entity objects of the following Tic-tac-toe software:

Tic-tac-toe is a computer game in which a human player and the computer **make** alternative moves on a 3 X 3 square. A move consists of **marking** a previously unmarked square. A player who first **places** three consecutive marks along a straight line (i.e., along a row, column, or diagonal) on the square **wins**. As soon as either the human player or the computer **wins**, a message congratulating the winner should be **displayed**. If neither player manages to **get** three consecutive marks along a straight line, and all the squares on the board are filled up, then the game is **drawn**. The computer always **tries to win** a game.

By performing a grammatical analysis of this problem statement, it can be seen that nouns that have been underlined in the problem description and the actions as the italicized verbs. However, on closer examination synonyms can be eliminated from the identified nouns. The list of nouns after eliminating the synonyms are the following: Tic-tac-toe, computer game, human player, move, square, mark, straight line, board, row, column, and diagonal.

- From this list of possible objects, nouns can be eliminated like human player as it does not belong to the problem domain. Also, the nouns square, game, computer, Tic-tac-toe, straight line, row, column, and diagonal can be eliminated, as any data and methods can not be associated with them. The noun **move** can also be eliminated from the list of potential objects since it is an imperative verb and actually represents an action. Thus, there is only one object left – board.
- After experienced in object identification, it is not normally necessary to really identify all nouns in the problem description by underlining them or actually listing them down, and systematically eliminate the non-objects to arrive at the final set of objects.

Interaction Modelling

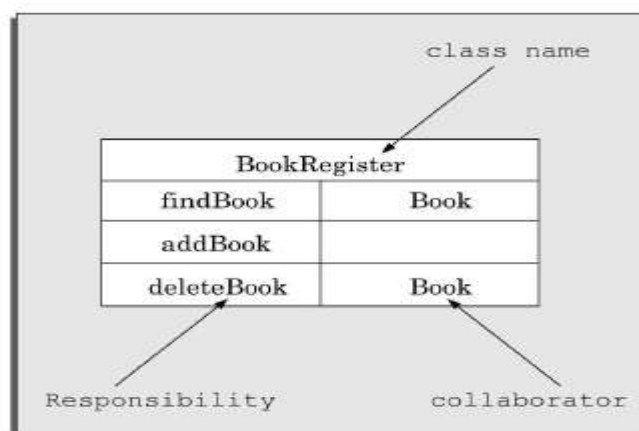
Identify two goals of interaction modeling. The primary goal of interaction modeling are the following:

- To allocate the responsibility of a use case realization among the boundary, entity, and controller objects. The responsibilities for each class is reflected as an operation to be supported by that class.
- To show the detailed interaction that occur over time among the objects associated with each use case.

The CRC cards technique.

- The interactions diagrams for only simple use cases that involve collaboration among a limited number of classes can be drawn from an inspection of the use case description.
- More complex use cases require the use of CRC cards where a number of team members participate to determine the responsibility of the classes involved in the use case realization.
- CRC (Class-Responsibility-Collaborator) technology was pioneered by Ward Cunningham and Kent Beck at the research laboratory of Tektronix at Portland, Oregon, USA. CRC cards are index cards that are prepared one per each class.
- On each of these cards, the responsibility of each class is written briefly. The objects with which this object needs to collaborate its responsibility are also written.
- CRC cards are usually developed in small group sessions where people role play being various classes. Each person holds the CRC card of the classes he is playing the role of. The cards are deliberately made small (4 inch ´ 6 inch) so that each class can have only limited number of responsibilities. A responsibility is the high level description of the part that a class needs to play in the realization of a use case. An example CRC card for the BookRegister class of the Library Automation System is shown in fig. 8.3.

After assigning the responsibility to classes using CRC cards, it is easier to develop the interaction diagrams by flipping through the CRC cards



OOD Goodness criteria

- It is quite obvious that there are several subjective judgments involved in arriving at a good object-oriented design.
- Therefore, several alternative design solutions to the same problem are possible. In order to be able to determine which of any two designs is better, some criteria for judging the goodness of a design must be identified.
- The following are some of the accepted criteria for judging the goodness of a design
 - **Coupling guidelines.** The number of messages between two objects or among a group of objects should be minimum. Excessive coupling between objects is detrimental to modular design and prevents reuse.
 - **Cohesion guideline.** In OOD, cohesion is about three levels:

Cohesiveness of the individual methods. Cohesiveness of each of

the individual method is desirable, since it assumes that each method does only a well-defined function.

Cohesiveness of the data and methods within a class. This is desirable since it assures that the methods of an object do actions for which the object is naturally responsible, i.e. it assures that no action has been improperly mapped to an object.

Cohesiveness of an entire class hierarchy. Cohesiveness of methods within a class is desirable since it promotes encapsulation of the objects.

- **Hierarchy and factoring guidelines.** A base class should not have too many subclasses. If too many subclasses are derived from a single base class, then it becomes difficult to understand the design. In fact, there should approximately be no more than 7 ± 2 classes derived from a base class at any level.
- **Keeping message protocols simple.** Complex message protocols are an indication of excessive coupling among objects. If a message requires more than 3 parameters, then it is an indication of bad design.
- **Number of Methods.** Objects with a large number of methods are likely to be more application-specific and also difficult to comprehend – limiting the possibility of their reuse. Therefore, objects should not have too many methods. This is a measure of the complexity of a class. It is likely that the classes having more than about seven methods would have problems.
- **Depth of the inheritance tree.** The deeper a class is in the class inheritance hierarchy, the greater is the number of methods it is likely to inherit, making it more complex. Therefore, the height of the inheritance tree should not be very large.
- **Number of messages per use case.** If methods of a large number of objects are invoked in a chain action in response to a single message, testing and debugging of the objects becomes complicated. Therefore, a single message should not result in excessive message generation and transmission in a system.

Response for a class. This is a measure of the maximum number of methods that an instance of this class would call. If the same method is called more than once, then it is counted only once. A class which calls more than about seven different methods is susceptible to errors

7. Explain Pattern?

PATTERN:

- The basic idea behind patterns is that if you can master a few important patterns, you can easily spot them in application development problems and effortlessly use the pattern solutions.
- Patterns can be viewed as helping designers to make certain important design decisions. At basic level, patterns can also be viewed as **well-documented building blocks** for software design.
- Patterns are created and documented by people who **spot repeating themes** across designs. Once patterns are created and documented, they can be used by different designers.
- A highly effective means to **capture and transfer design knowledge** across different design problems.
- A pattern documentation usually consists of **four important parts**:
 - The problem
 - The context in which the problem occurs
 - The solution
 - The context within which the solution would work and would not work

Types of Patterns:-

- Architectural patterns
- Design patterns
- Idioms

Architectural pattern

- It identifies and provides solutions to problems that are identifiable while carrying out architectural (or very high-level) designs.
- It concern **overall structure of software** system
- It cannot directly be translated to code, but **form the basis for more detailed design**.
- Architectural designs are usually **constructed for very large problems**.

Design patterns

- Suggests a scheme **for structuring the classes** in a design solution and defines the interactions required among those classes.
- Design pattern solutions are described in terms of classes, their instances, their roles and collaborations.

Idioms

- Idioms are low-level patterns that are **programming language-specific**
- An idiom describes how to implement a solution to a particular problem **using the features of a given programming language**.
- As soon as they see a requirement while working out a programming solution, the required idiom occurs to them instantly and relieves them from having **to select of the exact constructs to use in the program** and at the same time **improves the quality** of the program and reduces the bug detection and correction iterations.

Comparison of different types of patterns

- Architectural patterns are high-level strategies that concern the overall solutions to large-scale problems.
- Design patterns are solutions for specific parts of medium-scale problems and recommend certain structures and behavior of participating entities.
- Idioms are paradigm-specific and language-specific programming solutions.

Patterns versus algorithms

- Algorithms focus on solving problems with reduced space and/or time requirements, whereas patterns focus on understandability and maintainability of design and easier development

Pros and cons of design pattern

Advantages:

- Design patterns provide a common vocabulary that helps to improve communication among the developers
- Capture and disseminate expert knowledge
- Produce designs that are flexible, standard and easily maintainable
- Improve the designer productivity

Disadvantages:-

- Design patterns do not directly lead to code reuse
- At present no methodology is available that can be used to select the right design pattern at the right point during a design exercise.

Antipattern:

- It represents **lessons learned from a bad design**
- **Types of antipattern**
 - Those that describe bad solutions to problems which leads to bad situations
 - Those that describe how to avoid bad solutions to problems

Some design patterns:-

Expert Pattern

Problem: Which class should be responsible for doing certain things?

Solution: Assign responsibility to the information expert – the class that has the information necessary to fulfill the required responsibility. The expert pattern expresses the common intuition that objects do things related to the information they have. The class diagram and collaboration diagrams for this solution to the problem of which class should compute the total sales is shown in the fig. 8.1.



(a)

(b)

Fig. : Expert pattern: (a) Class diagram (b) Collaboration diagram

Creator Pattern

Problem: Which class should be responsible for creating a new instance of some class?

Solution: Assign a class C1 the responsibility to create an instance of class C2, if one or more of the following are true:

- C1 is an aggregation of objects of type C2.
- C1 contains objects of type C2.
- C1 closely uses objects of type C2.
- C1 has the data that would be required to initialize the objects of type C2, when they are created.

Explain controller pattern and circumstances when it can be used.

Controller Pattern:

Problem: Who should be responsible for handling the actor requests?

Solution: For every use case, there should be a separate controller object which would be responsible for handling requests from the actor. Also, the same controller should be used for all the actor requests pertaining to one use case so that it becomes possible to maintain the necessary information about the state of the use case. The state information maintained by a controller can be used to identify the out-of-sequence actor requests, e.g. whether voucher request is received before arrange payment request.

Explain facade pattern and circumstances when it can be used.

Façade Pattern:

Problem: How should the services be requested from a service package?

Context in which the problem occurs: A package as already discussed is a cohesive set of classes – the classes have strongly related responsibilities. For example, an RDBMS interface package may contain classes that allow one to perform various operations on the RDBMS.

Solution: A class (such as DBfacade) can be created which provides a common interface to the services of the package.

Explain model view separation pattern and circumstances when it can be used.

Model View Separation Pattern:

Problem: How should the non-GUI classes communicate with the GUI classes?

Context in which the problem occurs: This is a very commonly occurring pattern which occurs in almost every problem. Here, model is a synonym for the domain layer objects, view is a synonym for the presentation layer objects such as the GUI objects.

Solution: The model view separation pattern states that model objects should not have direct knowledge (or

be directly coupled) to the view objects. This

means that there should not be any direct calls from other objects to the GUI objects. This results in a good solution, because the GUI classes are related to a particular application whereas the other classes may be reused.

There are actually two solutions to this problem which work in different circumstances as follows:

Solution 1: Polling or Pull from above

It is the responsibility of a GUI object to ask for the relevant information from the other objects, i.e. the GUI objects pull the necessary information from the other objects whenever required.

This model is frequently used. However, it is inefficient for certain applications. For example, simulation applications which require visualization, the GUI objects would not know when the necessary information becomes available. Other examples are, monitoring applications such as network monitoring, stock market quotes, and so on. In these situations, a “push-from-below” model of display update is required. Since “push-from-below” is not an acceptable solution, an indirect mode of communication from the other objects to the GUI objects is required.

Solution 2: Publish- subscribe pattern

An event notification system is implemented through which the publisher can indirectly notify the subscribers as soon as the necessary information becomes available. An event manager class can be defined which keeps track of the subscribers and the types of events they are interested in. An event is published by the publisher by sending a message to the event manager object. The event manager notifies all registered subscribers usually via a parameterized message (called a callback). Some languages specifically support event manager classes. For example, Java provides the `EventListener` interface for such purposes.

Explain intermediary pattern (i.e. proxy pattern) and circumstances when it can be used.

Intermediary Pattern or Proxy

Problem: How should the client and server objects interact with each other?

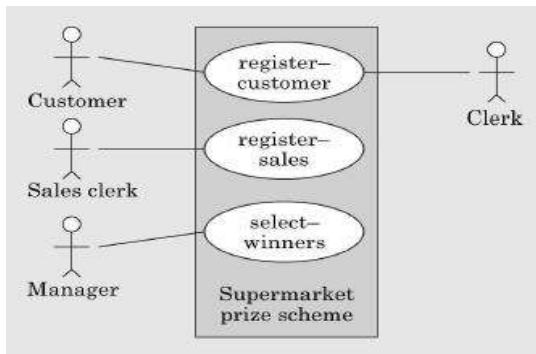
Context in the problem occurs: The client and server terms as used here refer to software components existing across a network. The clients are consumers of services provided by the servers.

Solution: A proxy object at the client side can be defined which is a local sit-in for the remote server object. The proxy hides the details of the network transmission. The proxy is responsible for determining the server address, communicating the client request to the server, obtaining the server response and seamlessly passing that to the client. The proxy can also augment (or filter) information that is exchanged between the client and the server. The proxy could have the same interface as the remote server object so that the client feels as if it is interacting directly with the remote server object and the complexities of network transmissions are abstracted out.

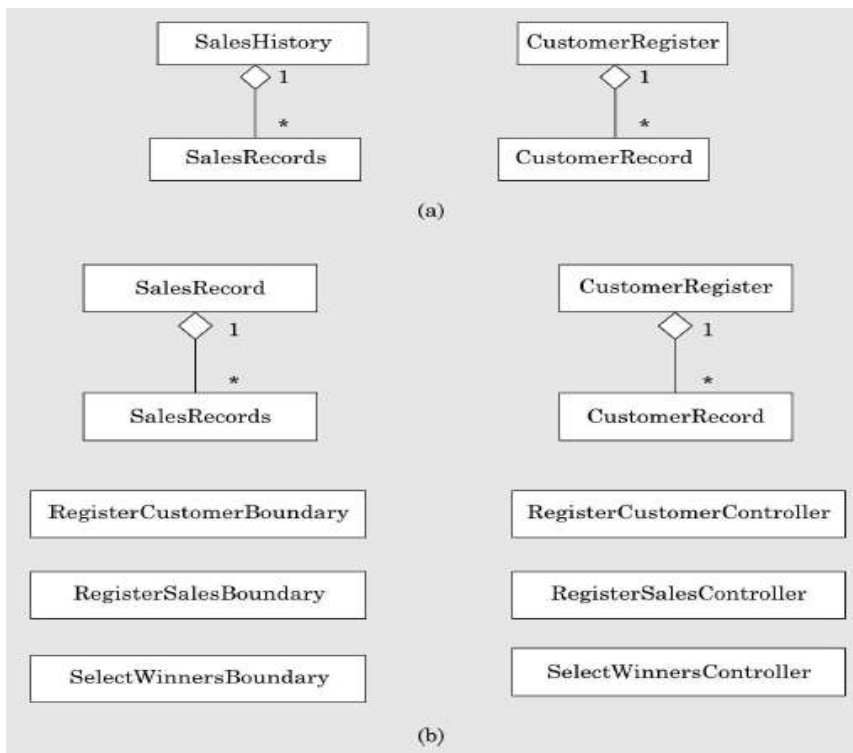
8.Draw UML diagrams for Supermarket scheme

Consider the supermarket prizes scheme software. The step-by-step analysis and design workout of this problem is as follows:

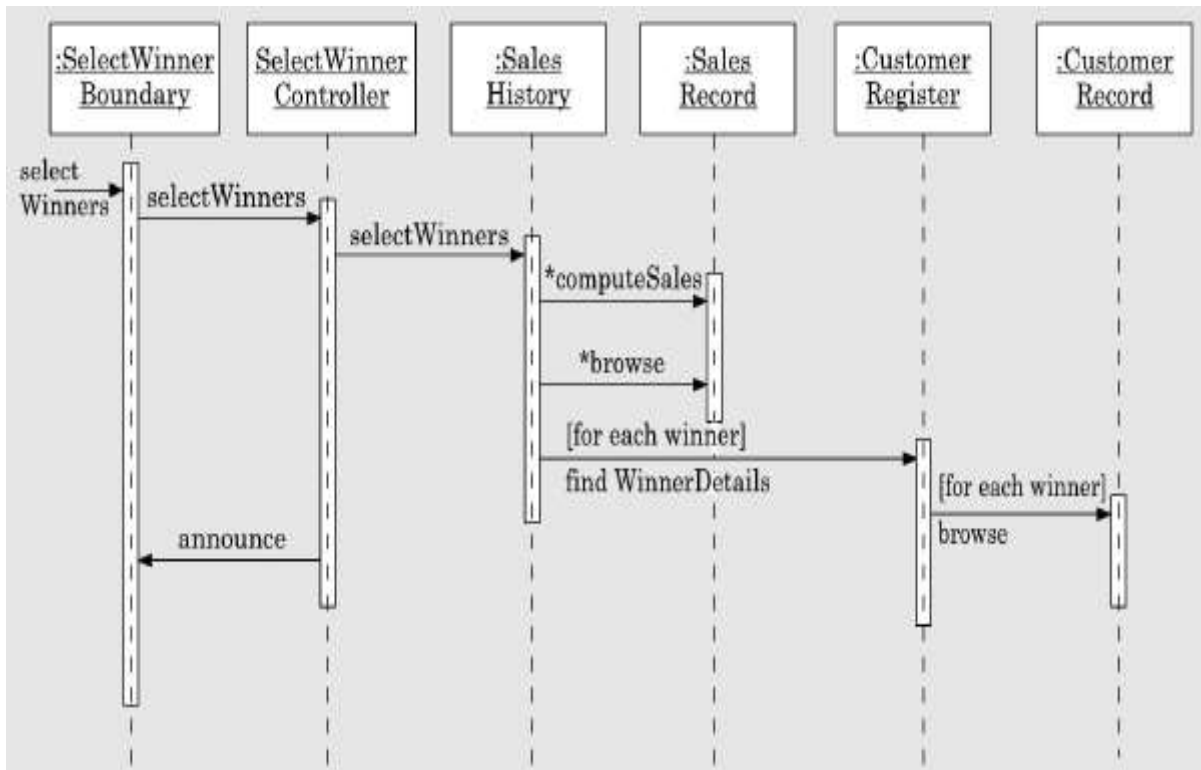
- The use case model



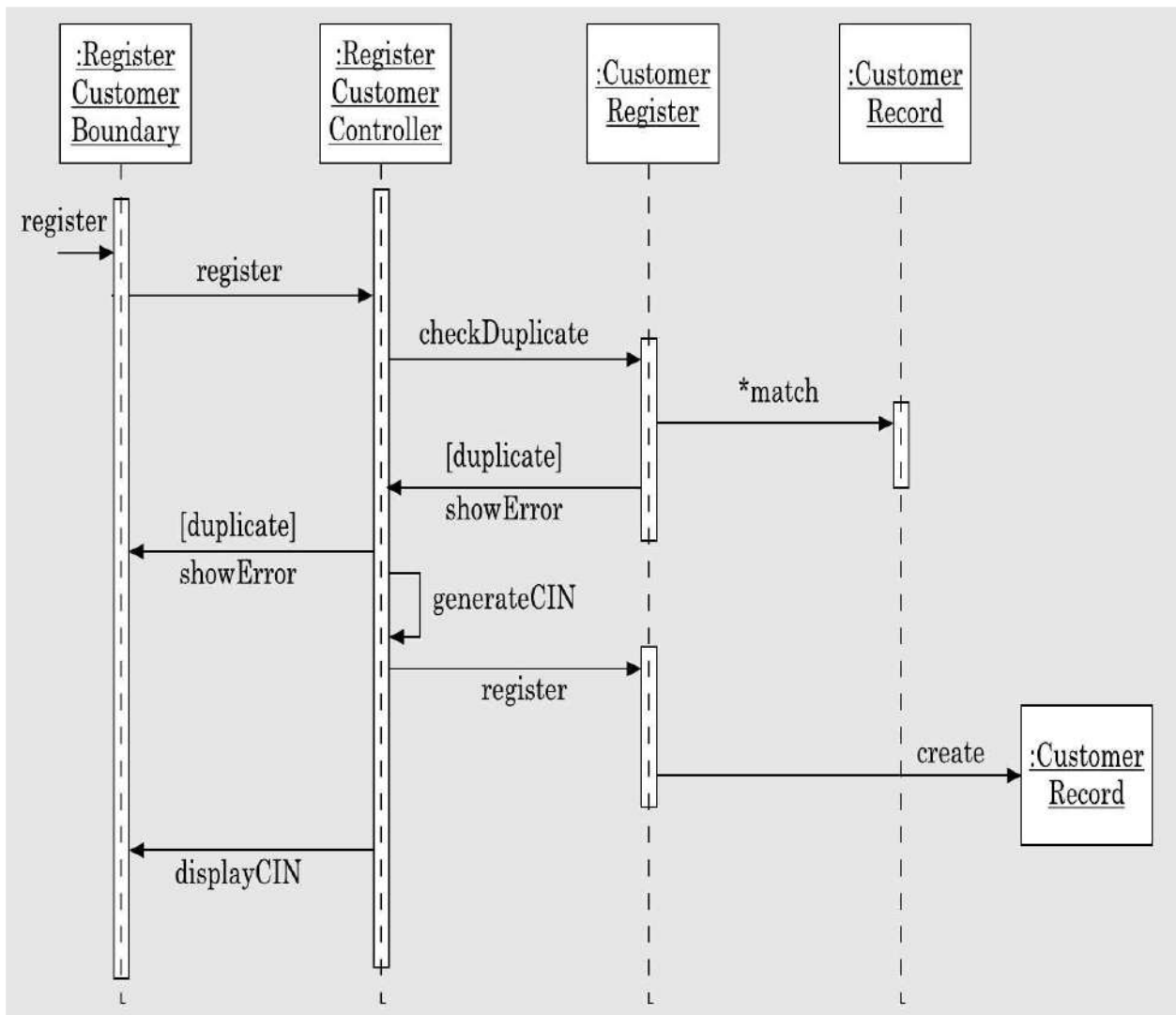
- a) Initial domain model
- b) Refined domain model



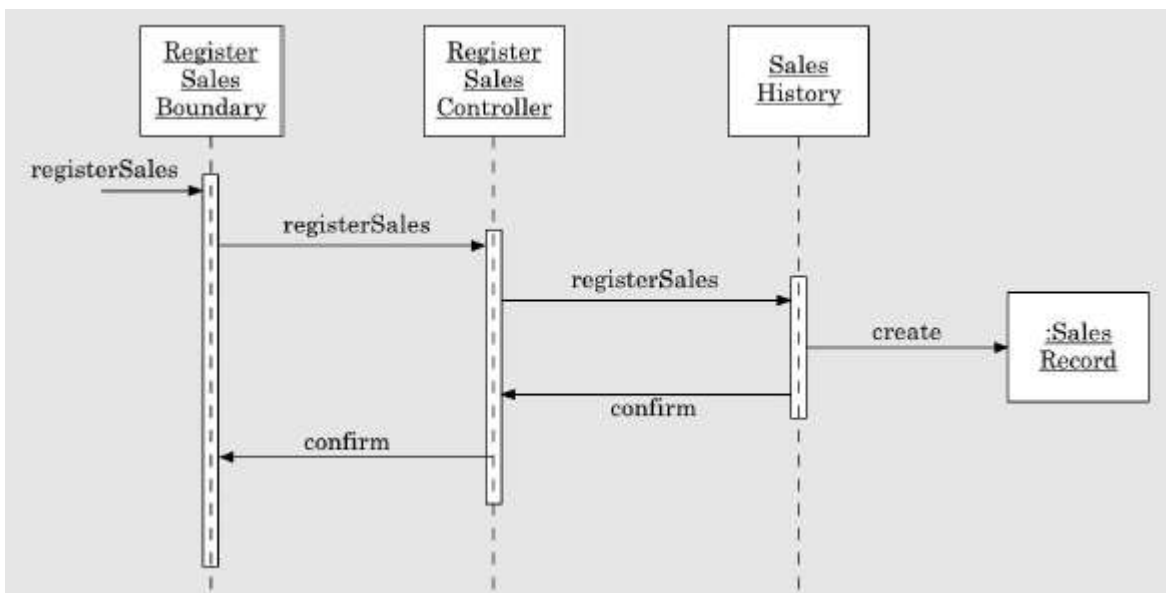
- Sequence diagram for select winner list use case



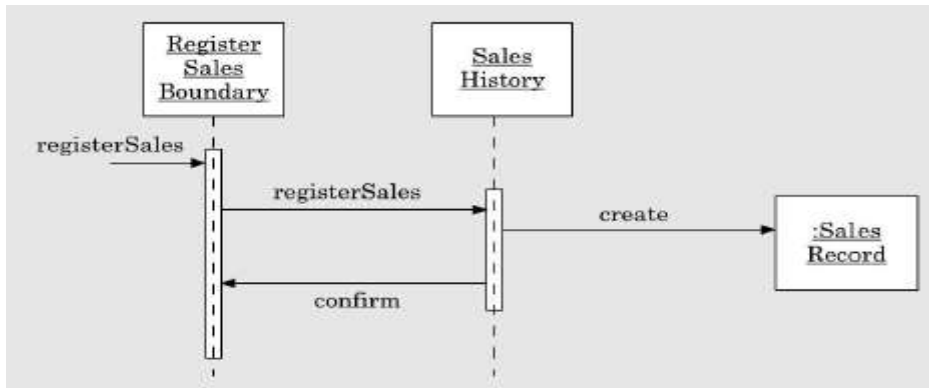
- Sequence diagram for the register customer use case



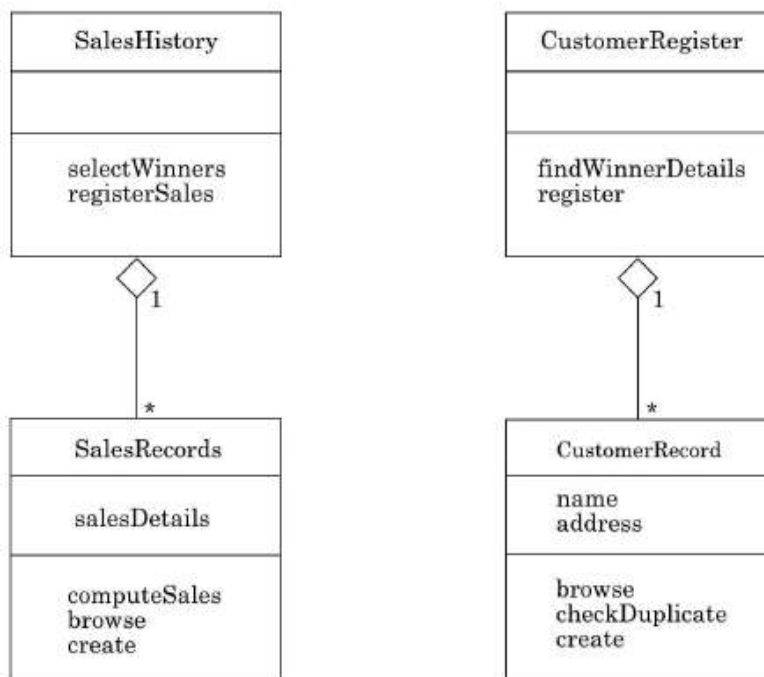
- Sequence diagram for the register sales use case



- Refined Sequence diagram for the register sales use case



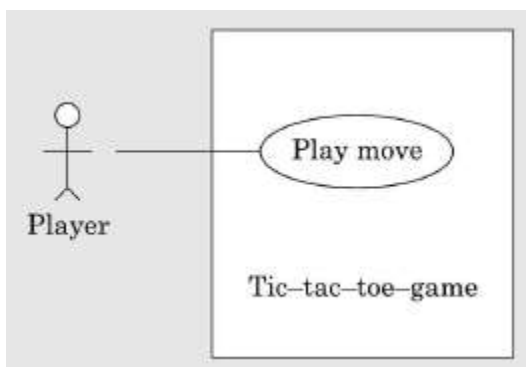
- Class diagram



9. Draw UML diagrams for Tic-tac-toe computer game

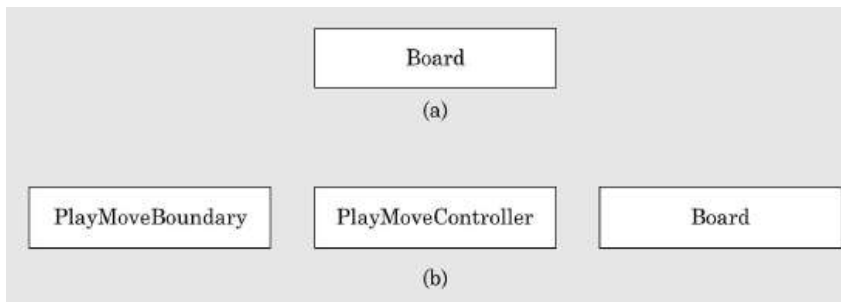
Consider the Tic-tac-toe computer game.

- The use case model

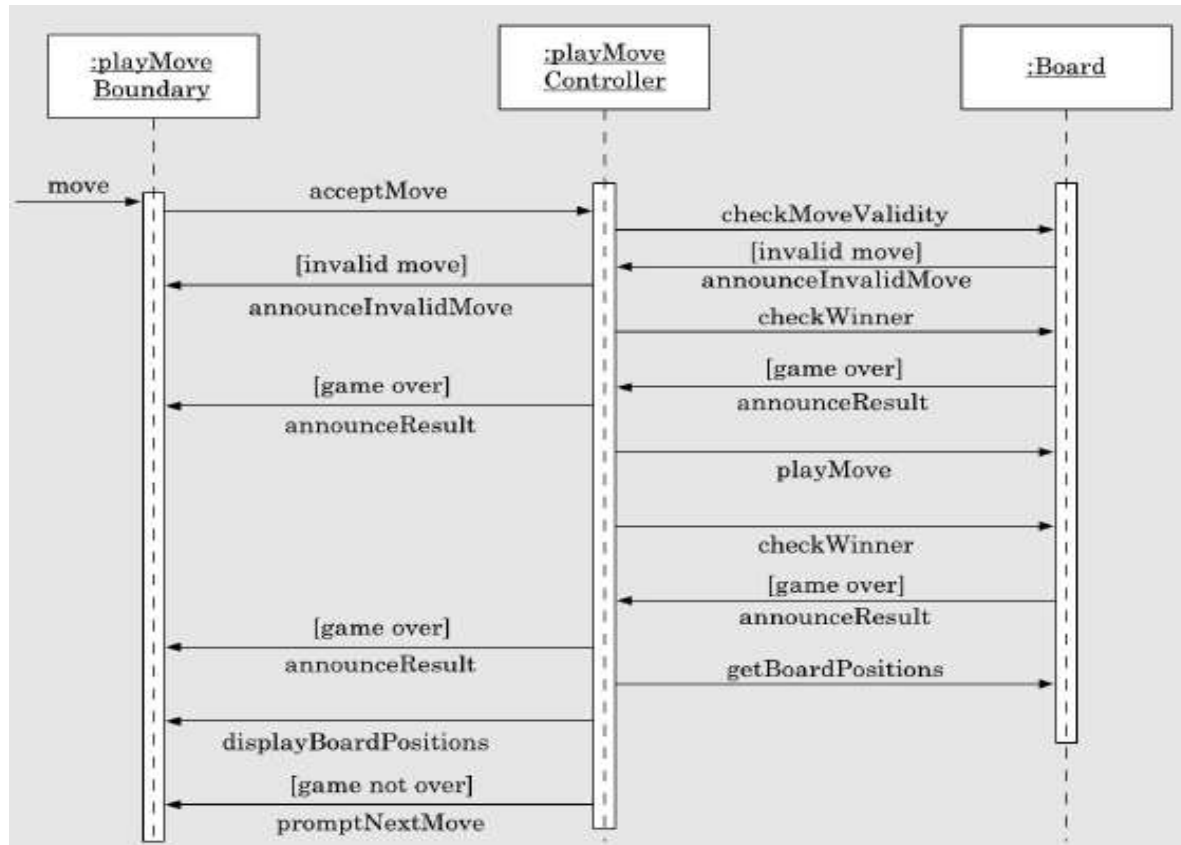


- a) Initial domain model

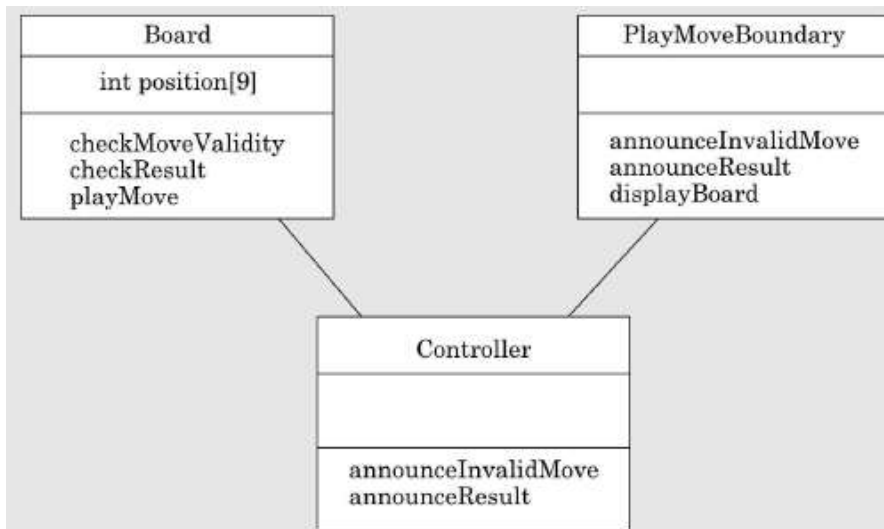
b) Refined domain model



- Sequence diagram for the play move use case



- Class diagram



10. Write short notes on

- a) Use case diagram**
- b) Class diagram**
- c) Interaction diagram (Nov 2017)**

Refer question number : 2, 3, 4 Page number : 20